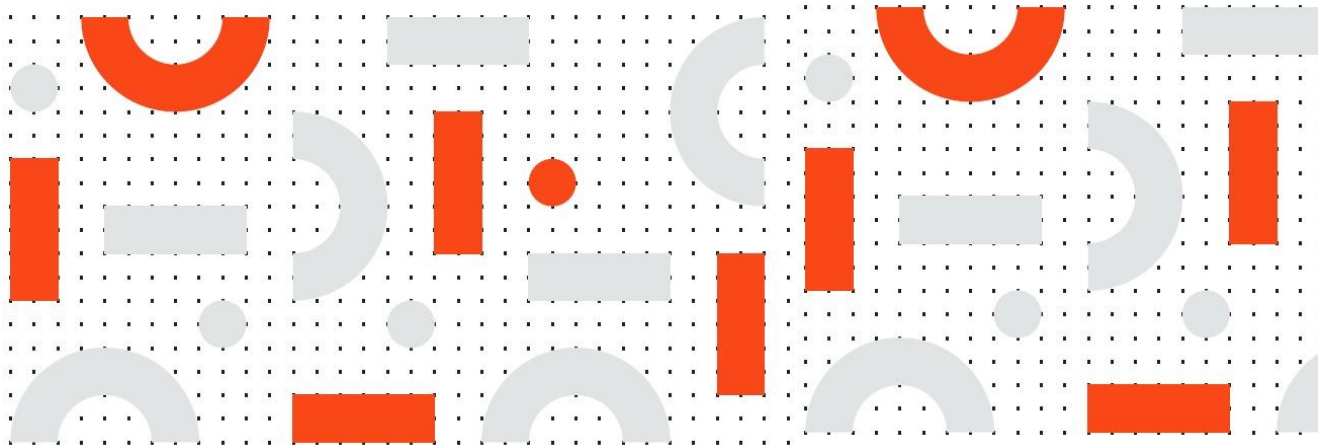
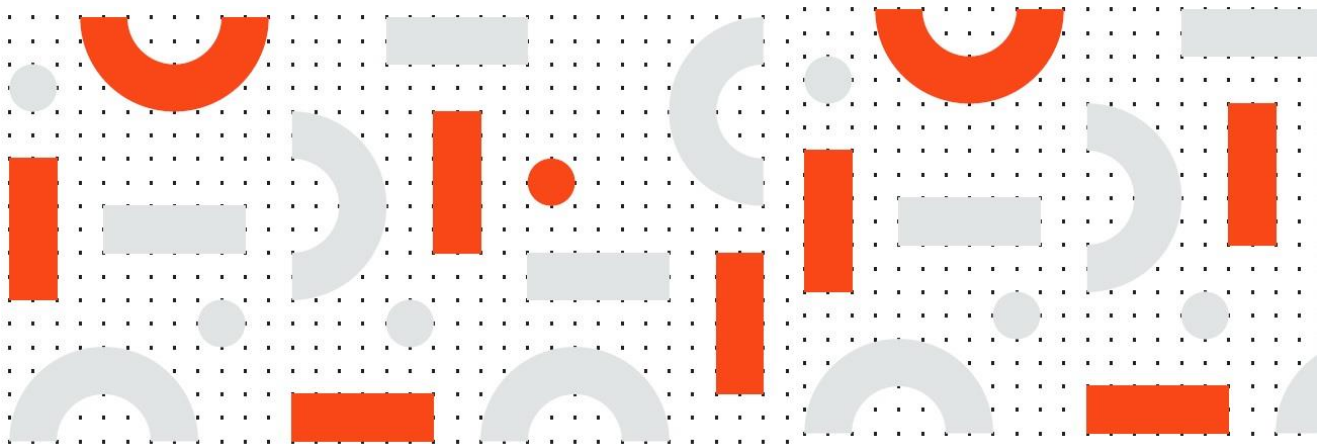




# RPA Design and Development Course

---

LAB EXERCISE MANUAL v1.1



**Academic  
Alliance**

# Table of Contents

|           |                                                                        |           |
|-----------|------------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>UiPath Studio.....</b>                                              | <b>3</b>  |
| 1.1       | Downloading & installing UiPath Studio.....                            | 3         |
| 1.2       | Installing browser extensions .....                                    | 10        |
| 1.2.1     | Installing extension on Chrome browser .....                           | 11        |
| 1.2.2     | Installing extension for Firefox browser.....                          | 13        |
| 1.2.3     | Installing extension for Edge browser .....                            | 14        |
| <b>2</b>  | <b>Number series .....</b>                                             | <b>15</b> |
| 2.1       | Natural number series.....                                             | 15        |
| 2.2       | Odd number series .....                                                | 18        |
| 2.3       | Even number series .....                                               | 21        |
| 2.4       | Prime number series.....                                               | 24        |
| 2.5       | Number order sorting.....                                              | 29        |
| <b>3</b>  | <b>Variable swapping.....</b>                                          | <b>33</b> |
| 3.1       | Using three bucket method.....                                         | 33        |
| 3.2       | Using two variables only.....                                          | 37        |
| <b>4</b>  | <b>Print “Hello” .....</b>                                             | <b>41</b> |
| 4.1       | Print "Hello" by using Sequence activity .....                         | 41        |
| 4.2       | Print "Hello" by using Flowchart activity .....                        | 43        |
| <b>5</b>  | <b>Addition of two numbers.....</b>                                    | <b>45</b> |
| <b>6</b>  | <b>Displaying a Sun Sign.....</b>                                      | <b>48</b> |
| <b>7</b>  | <b>Guessing game.....</b>                                              | <b>53</b> |
| <b>8</b>  | <b>Compare two columns of a spreadsheet .....</b>                      | <b>56</b> |
| <b>9</b>  | <b>Disk cleanup .....</b>                                              | <b>60</b> |
| <b>10</b> | <b>Extracting data from a website.....</b>                             | <b>63</b> |
| <b>11</b> | <b>Filling a webform from an excel sheet.....</b>                      | <b>66</b> |
| <b>12</b> | <b>Extracting data from an invoice image.....</b>                      | <b>72</b> |
| <b>13</b> | <b>Filling a webform from a true PDF file .....</b>                    | <b>76</b> |
| <b>14</b> | <b>Creating list of unique words.....</b>                              | <b>80</b> |
| <b>15</b> | <b>Extracting and storing the subject of emails.....</b>               | <b>85</b> |
| <b>16</b> | <b>Saving attachment with subject ‘Resume’ from unread emails.....</b> | <b>90</b> |
| <b>17</b> | <b>Checking data mismatch using “Try catch” mechanism .....</b>        | <b>93</b> |

**18 Generate monthly expenditure report**.....97

18.1 Open Outlook (Sequence).....98

18.2 Dispatcher..... 100

18.3 Process ..... 103

18.4 Close Outlook..... 104

# 1 UiPath Studio

## 1.1 Downloading & installing UiPath Studio

**Objective:** Step by Step guide explaining how to download and install UiPath Studio.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ **Downloading and installing UiPath Studio on your system.**

## Step by Step process:

**Step 1:** Open the link in browser - <http://www.uipath.com/academic-alliance-sw>

**Step 2:** Fill in the form and Request Community Edition.

**UiPath™**

START YOUR AUTOMATION JOURNEY

**UiPath Academic Alliance**  
prepares you with automation skills for a lifetime

- ✓ Download the fully featured UiPath Community Edition
- ✓ Free for students and educators enrolled in the UiPath Academic Alliance RPA classes

**Thank you for your interest in the UiPath Academic Alliance!**

To start your journey, download UiPath Community Edition from this page. It's free for students and educators, and comes with support via the [Community Forum](#), Guides, and other [resources](#). Please note, the Community Edition does not come with phone or email support.

Your school / institution has agreed to licensing terms as part of the Academic Alliance agreement.

If you are a student / professor whose school is not yet offering Robotic Process Automation (RPA) classes, you may still access the Community Edition for free [here](#).

To encourage your school to bring RPA to your campus, please refer them [here](#).

Fill in the form to get your copy:

FIRST NAME\* LAST NAME\*

EMAIL\*

COUNTRY\*

YOUR ACADEMIC INSTITUTION/ COLLEGE/ UNIVERSITY\*

ACADEMIC ROLE\*

☐ STUDENT

☐ EDUCATOR

FIELD OF STUDY\*

RPA COURSE START RPA COURSE END

TWITTER USERNAME FACEBOOK PROFILE URL LINKEDIN PROFILE URL

☐ I would like to receive communications about UiPath tailored to my interests and preferences, including latest news about products, services, events and promotions. For more information, please see our [Privacy Policy](#)

☐ I declare that I have read and I agree with the [License Agreement](#)\*

**Request Community Edition**

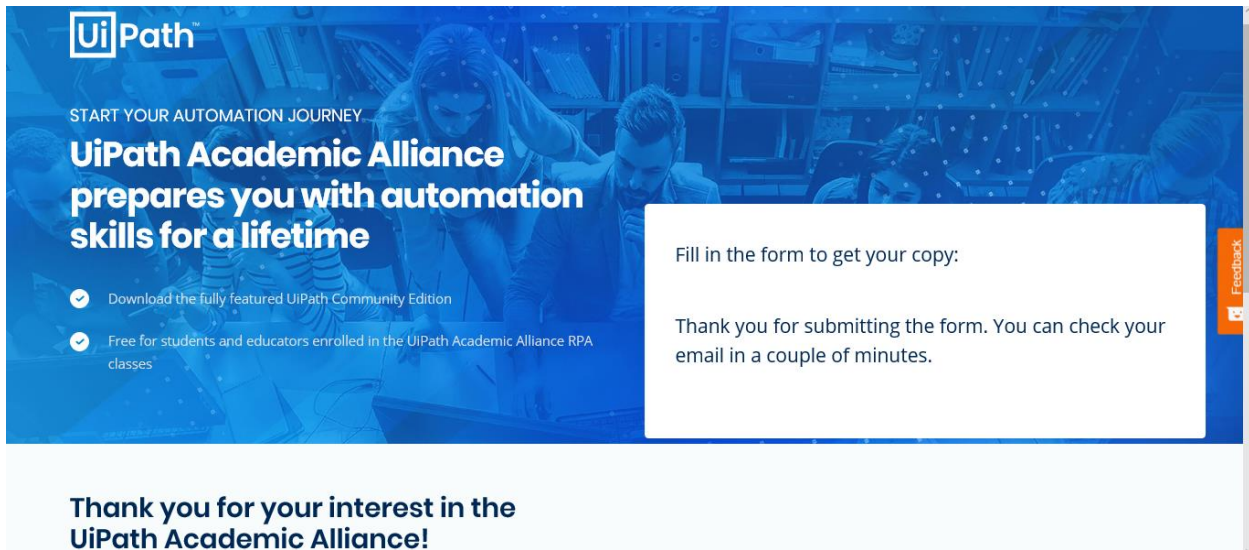
English

[Terms of Use](#) · [Privacy Policy](#) · [Cookies Policy](#) · [Personal Data Request](#) · [Business Partner Code](#)

© 2005-2019 UiPath. All rights reserved.

[in](#) [f](#) [t](#) [u](#) [r](#) [s](#)

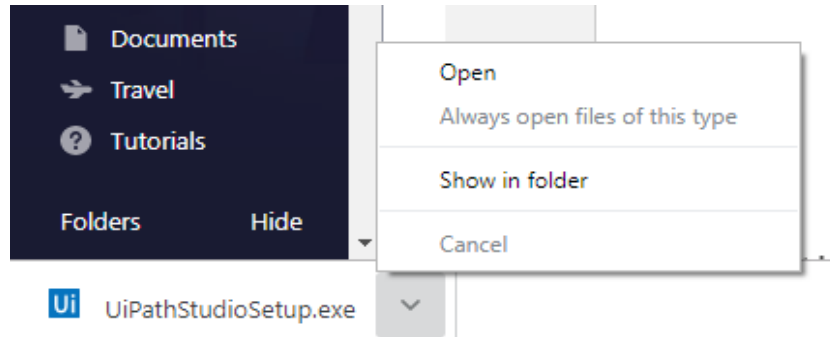
**Step 3:** Check your email in a couple of minutes.



**Step 4:** Open the Email received from [info@uipath.com](mailto:info@uipath.com) and click on 'Download the UiPath Community Edition'. UiPathStudioSetup.exe will start downloading in your task bar.

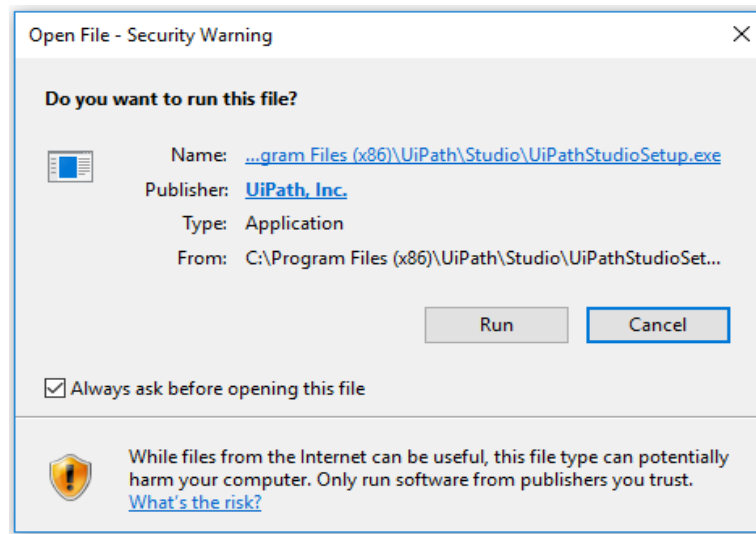
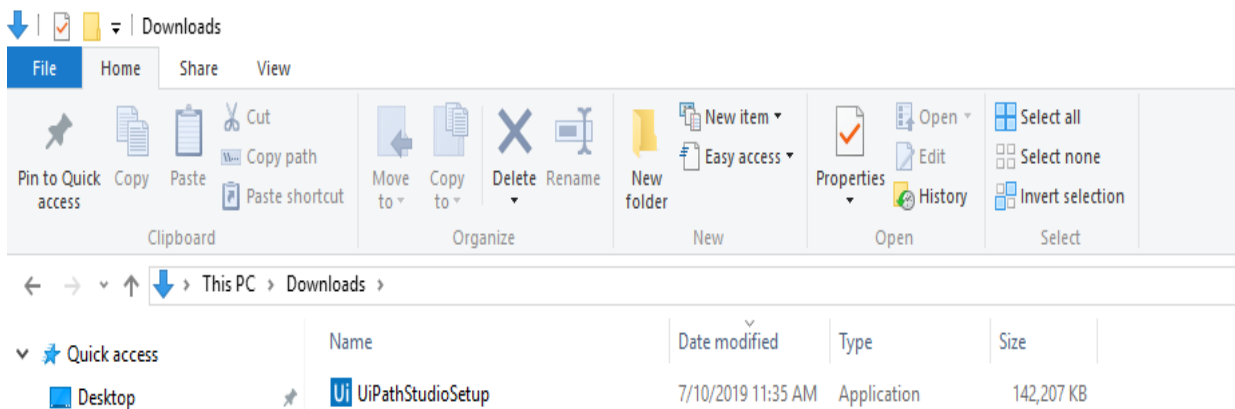


**Step 5:** Right click on UiPathStudioSetup.exe once completed and click on Show in folder.

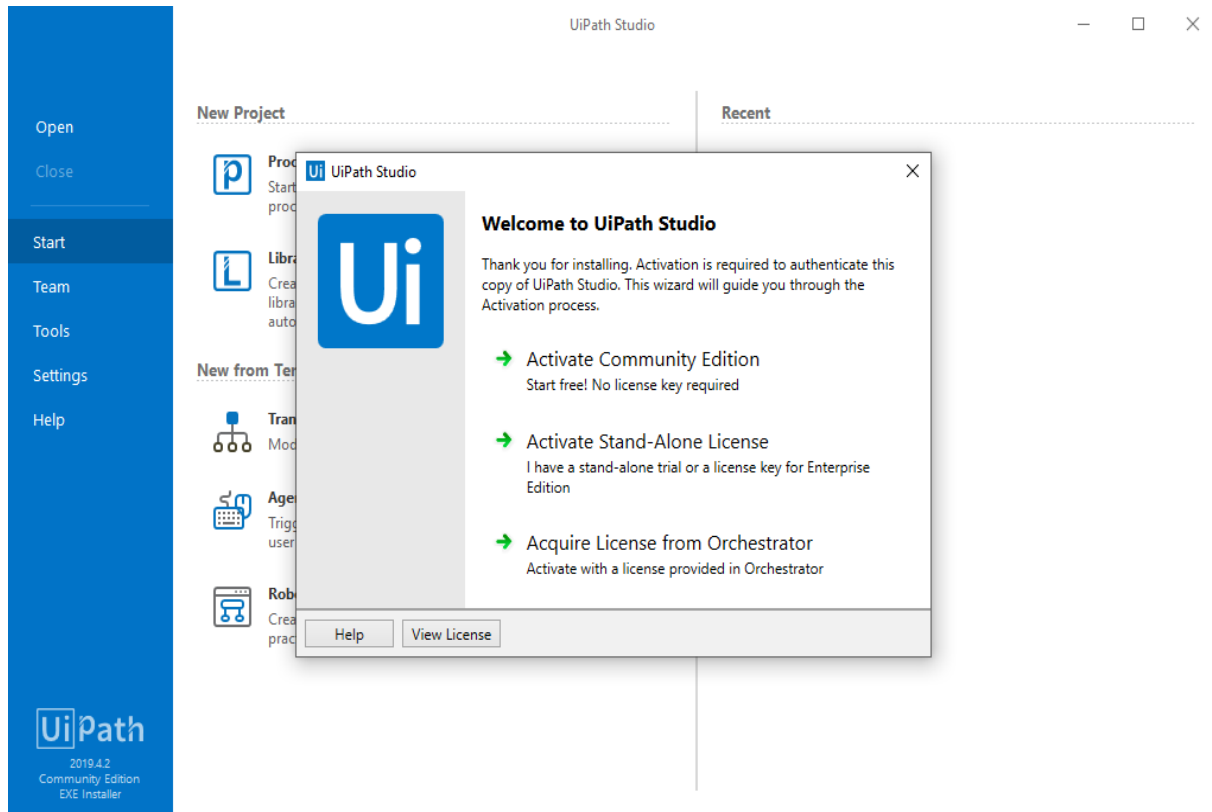


**Step 6:** Double click on **UiPathStudioSetup** and click on Run.

Default Path for UiPath Studio Setup - C:\Program Files (x86)\UiPath\Studio

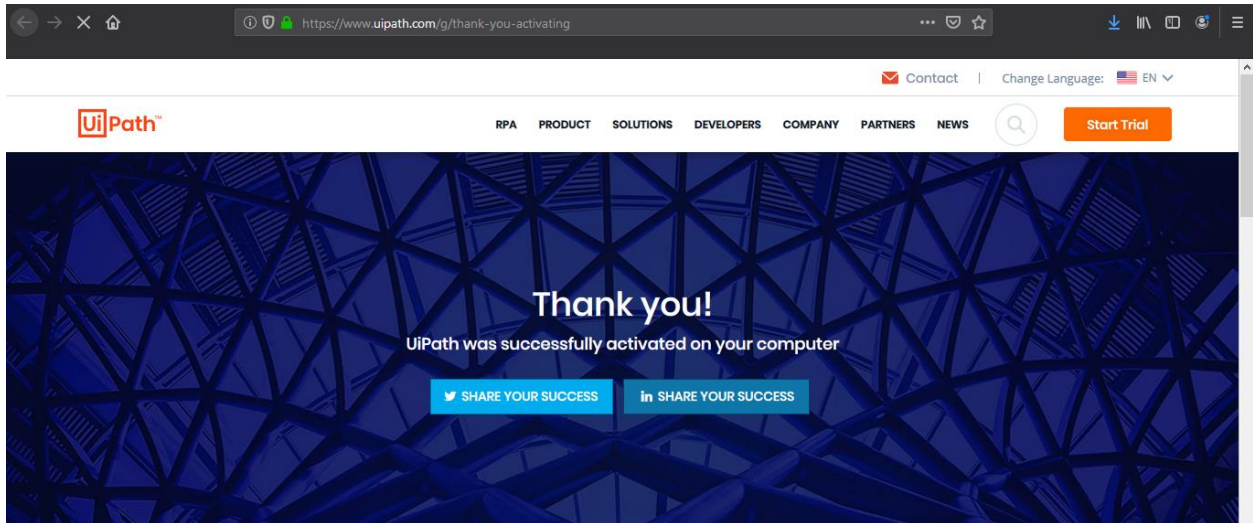


**Step 7:** UiPath Studio will open. Click on Activate Community Edition.

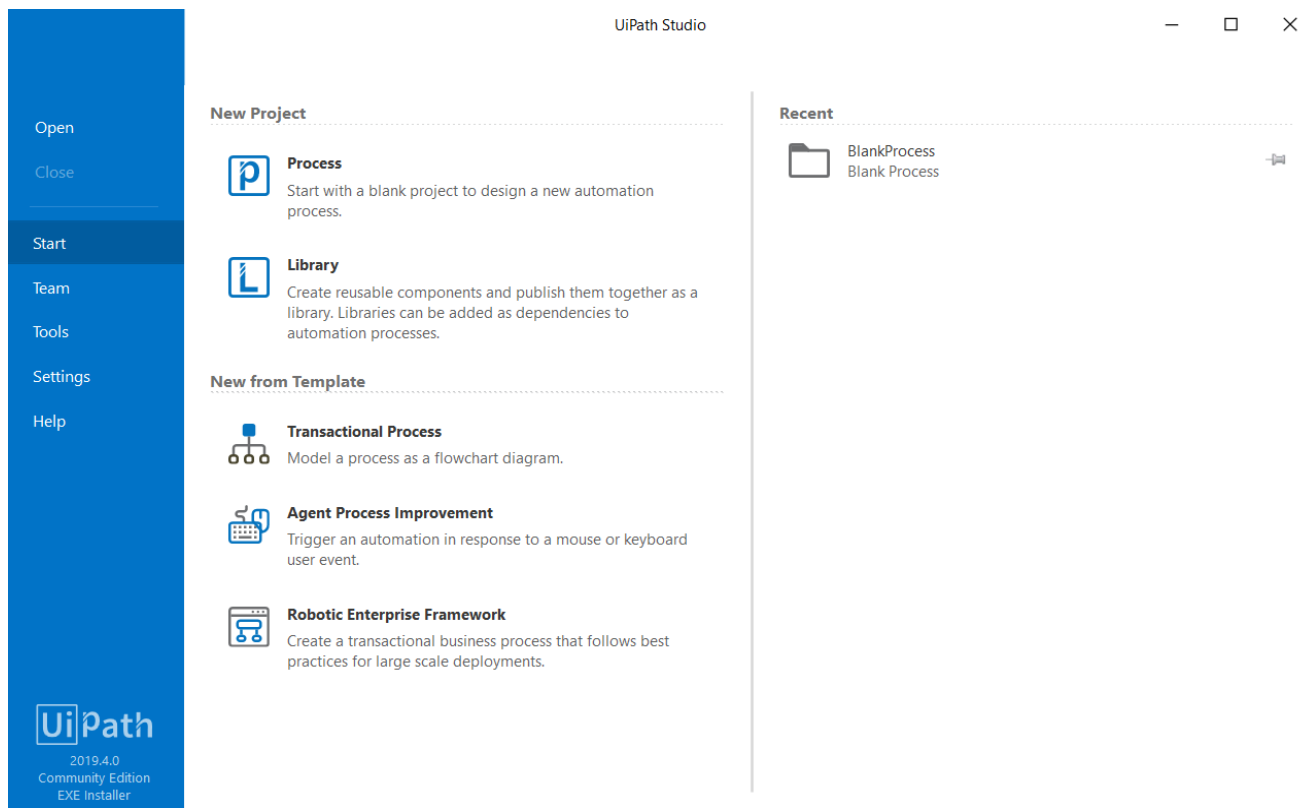


**Step 8:** Enter your Email Address used to create account on UiPath and click Activate.

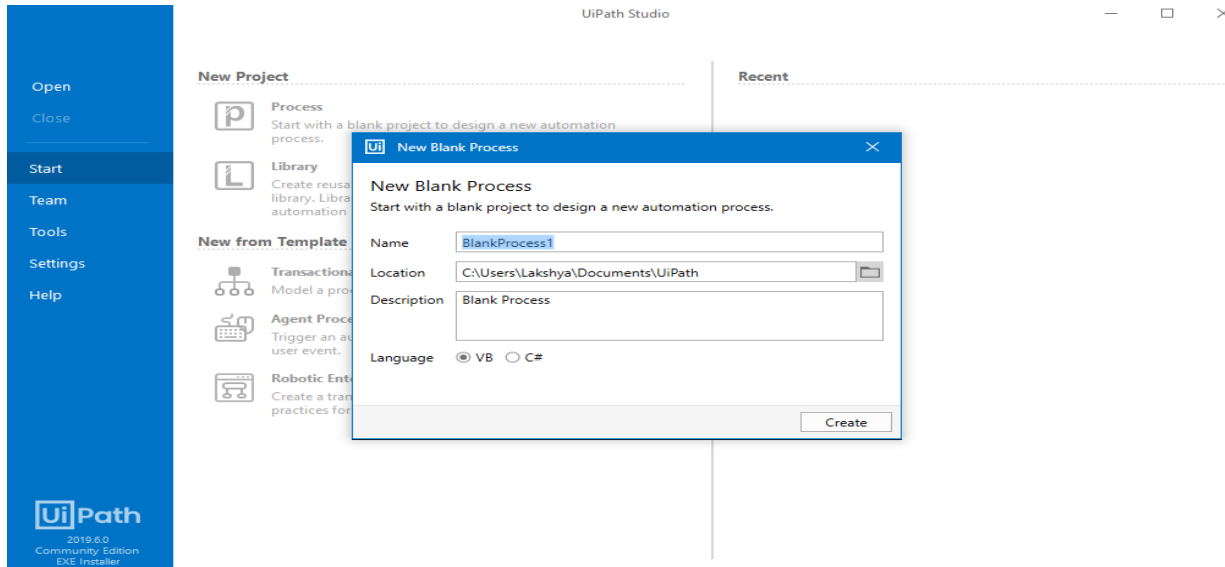
**Step 9:** “Thank you!” Message will be prompted after UiPath is successfully activated on your computer.



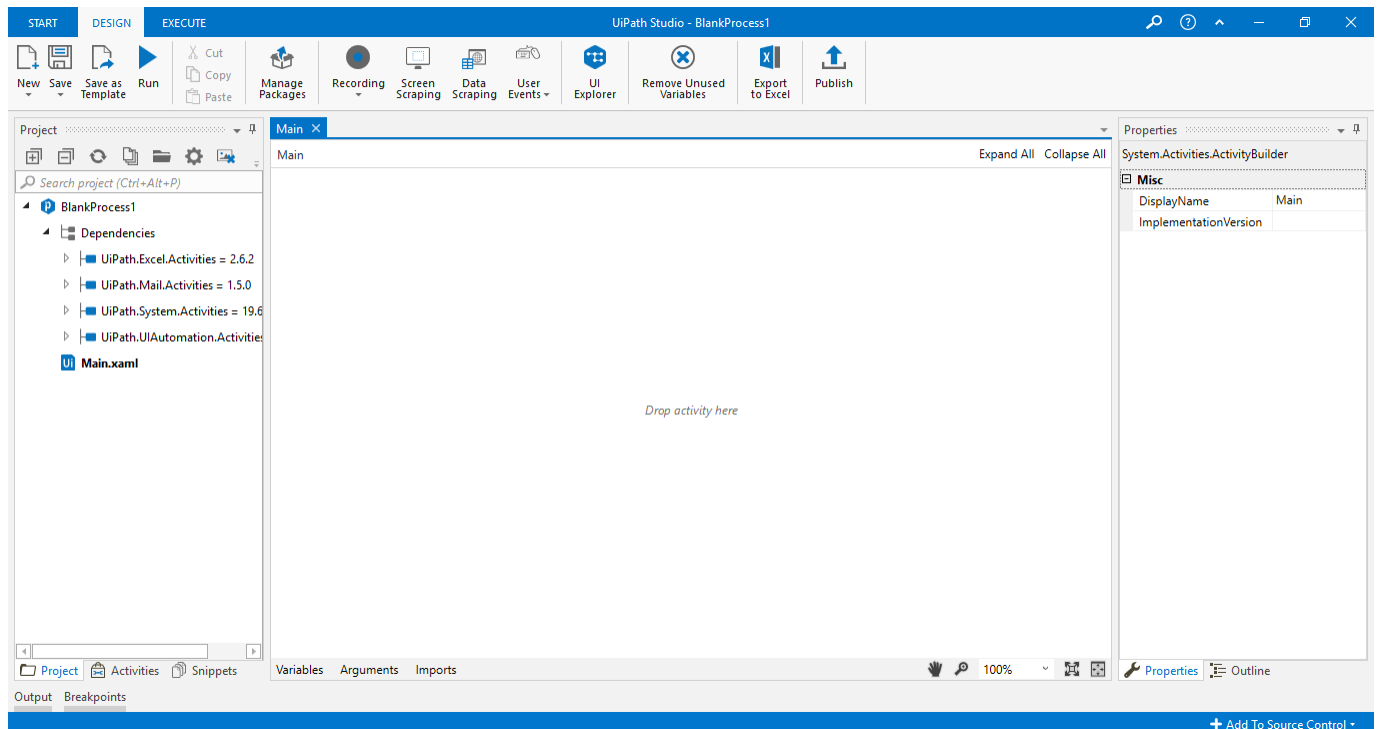
**Step 10:** Open UiPath Studio and you can explore the options available.



**Step 11:** Enter the Name, Location and Description of the process which you want to create.



**Step 12:** Finally, you can start working on your workflow



## 1.2 Installing browser extensions

**Objective:** Step by Step guide explaining how to install the below mentioned browser extensions in UiPath Studio.

1. Chrome
2. Firefox
3. Edge

### Learning Outcomes

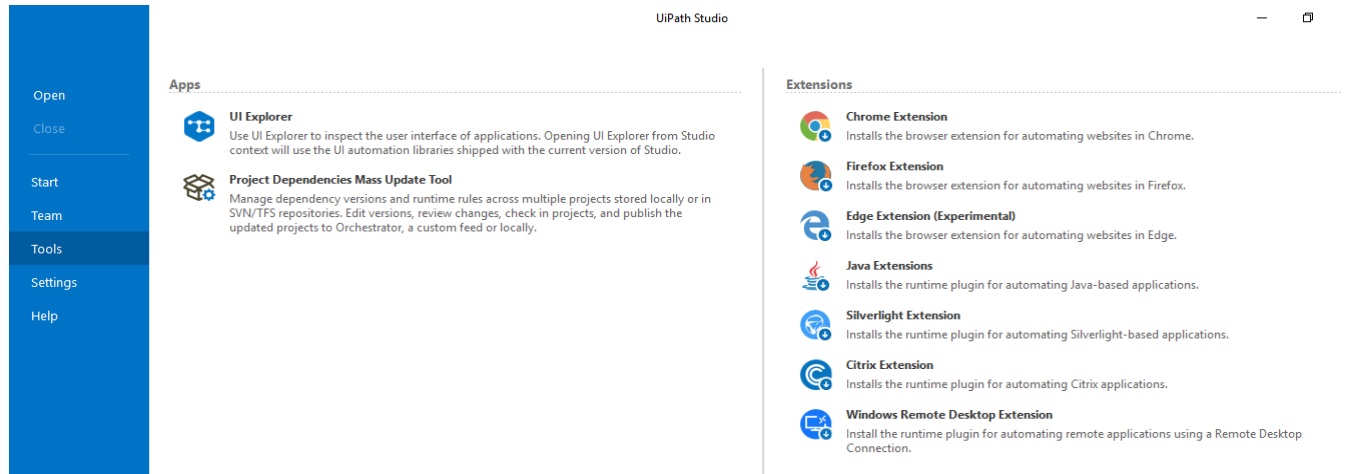
After completion of this exercise you will get familiar with the following:

- ✓ **Installing UiPath Browser extensions on Chrome and Firefox browser.**

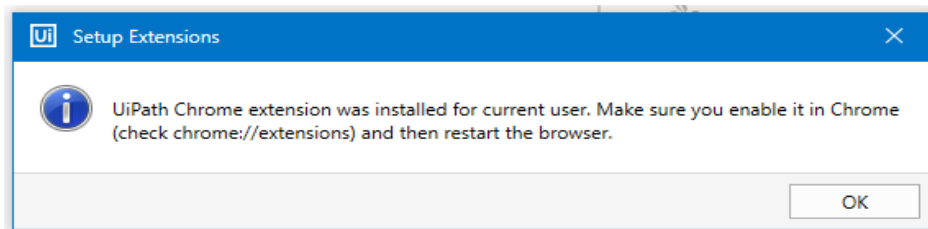
## 1.2.1 Installing extension on Chrome browser

### Step by Step process:

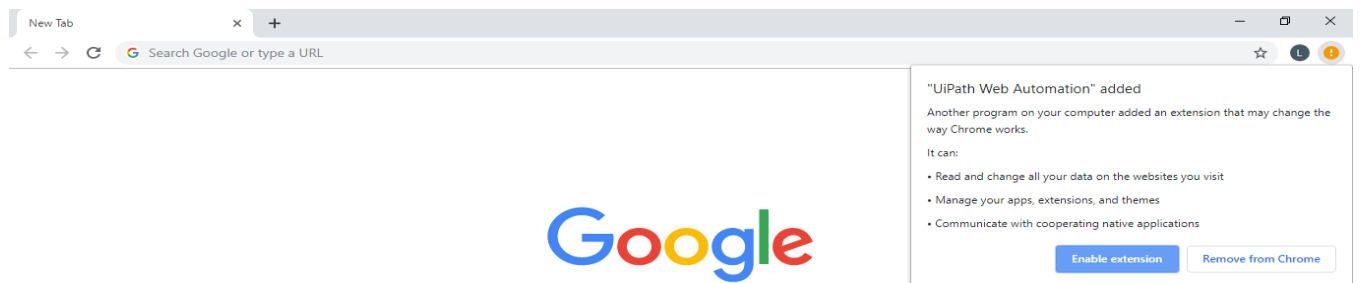
**Step 1:** Open UiPath Studio and select **Tools**. The extensions you can install become visible.



**Step 2:** Click **Chrome Extension**. A confirmation dialog box is displayed. Click OK.

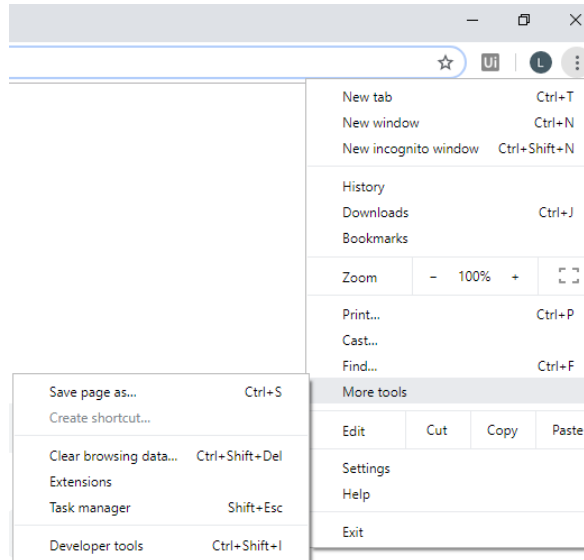


**Step 3:** Open Google Chrome and click the **Enable extension** button. Chrome extension is installed in system.

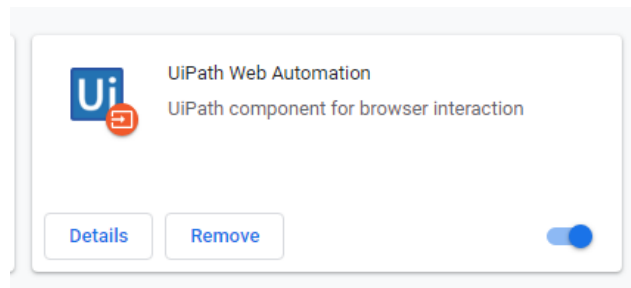


**File access will be disabled by default. In order to enable it, follow the steps below:**

**Step 1: Click Side Navigation Bar > More Tools > Extensions.**



**Step 2: The chrome://extensions/ page is displayed. In the chrome://extensions/ page, navigate to the UiPath extension. Click the Details button.**



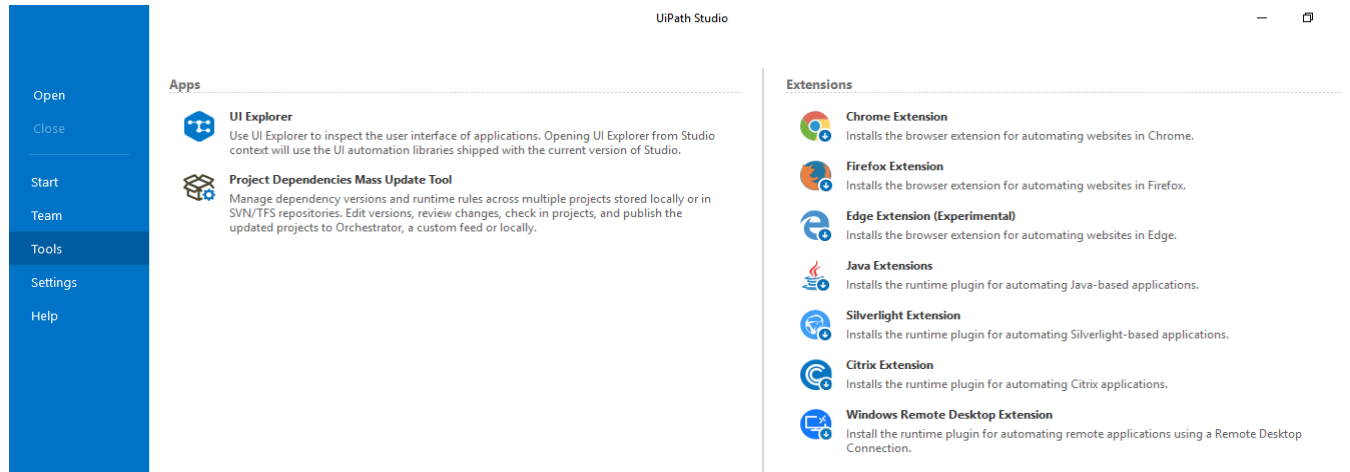
**Step 3: The UiPath Chrome extension details page is displayed. On this page, select the Allow access to file URLs check box.**



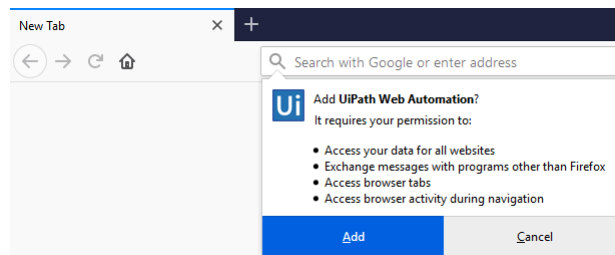
## 1.2.2 Installing extension for Firefox browser

### Step by Step process:

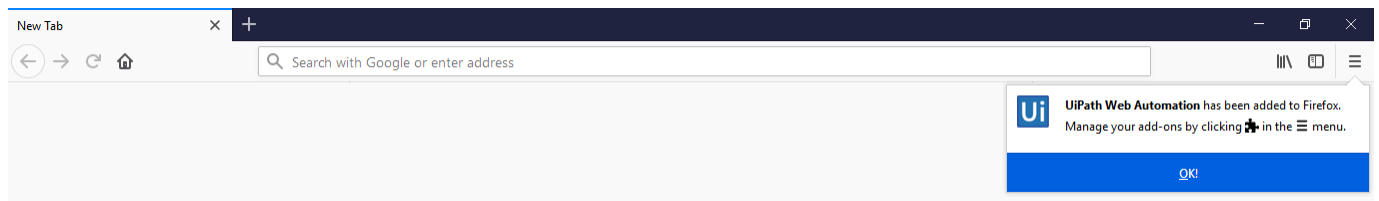
**Step 1:** Open UiPath Studio and select **Tools**. The extensions you can install become visible.



**Step 2:** Click **Firefox Extension**. Mozilla Firefox opens up and a pop-up is displayed.



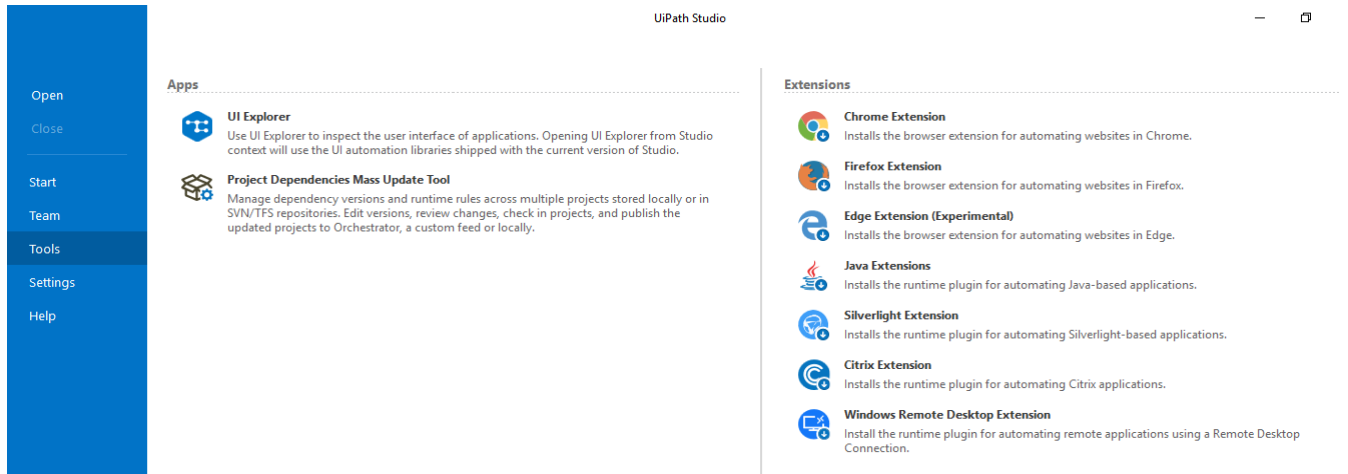
**Step 3:** Click the **Add** button, and then **OK** to confirm. The UiPath Firefox extension is installed in system.



### 1.2.3 Installing extension for Edge browser

#### Step by Step process:

**Step 1:** Open UiPath Studio and select **Tools**. The extensions you can install become visible.



**Step 2:** Click **Edge Extension**. A confirmation dialog box is displayed. Edge extension is installed in system.

## 2 Number series

### 2.1 Natural number series

**Objective:** To code a Robot in UiPath Studio to generate a series of natural numbers from 1 to 100.

#### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Write Line” activity and how to set **variables** in a code by using “Variable” panel.
- ✓ “While” activity and how to set conditions.
- ✓ Display output in “Output” panel.

#### Algorithm:

**Step 1:** START

**Step 2:** Declare a variable 'Number' and set Default Value as 0

**Step 3:** While 'Number' < 100

**Step 4:** Number = Number + 1

**Step 5:** STOP

## Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘This is the code to generate number series from 1 to 100’**.

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘// To generate a number series from 1 to 100 by using “While loop” activity.**

**Step: 6:** Drag and drop the **“Write line”** activity from the activity panel and name it as **Write line – ‘To display the text’** and inside the text field write **“The numbers series from 1 to 100 are:”**

**Step 7:** Create a variable from the **“Variables”** panel in the main sequence activity as under:

| Name   | Variable Type | Scope                                                                 | Default |
|--------|---------------|-----------------------------------------------------------------------|---------|
| Number | Int32         | Sequence – ‘This is the code to generate number series from 1 to 100’ | 0       |

**Step 8:** Drag and drop the **“While”** activity from the activity panel and name it as **While – ‘To check if the number is less than 100’**.

**Step 9:** Right-click on **“While”** activity select **Annotation > Add Annotation ‘This block of code will execute until the number printed is 100’**.

**Step 10:** Inside the **“While”** activity write the condition **“Number < 100”**.

**Step 11:** In the body section of **“While”** activity drag and drop the **“Sequence”** activity.

**Step 12:** Change the sequence name **Sequence – ‘Increase number by 1 and display the number series.’**

**Step 13:** Right-click on **“Sequence”** activity select **Annotation > Add Annotation ‘To Increase number by 1 and print the numbers in the output panel’**.

**Step 14:** Drag and drop the **“Assign”** activity inside the **“Sequence”** activity.

**Step 15:** Change the **“Assign”** activity name **Assign – ‘Add one to the previous value’**.

**Step 16:** Declare the '**Number**' variable in the **TO** box and '**Number+1**' in the value box.

**Step 17:** Drag and drop the "**Write line**" activity from the activity panel and name it as **Write line – 'Print numbers from 1 to 100'** and inside the text field write **Number.ToString**.

## 2.2 Odd number series

**Objective:** To code a Robot in UiPath Studio to generate a series of odd numbers from 1 to 100.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Write Line” activity and how to set **variables** in a code by using “Variable” panel.
- ✓ “While” activity and how to set conditions.
- ✓ Display output in “Output” panel.

### Algorithm:

**Step 1:** START

**Step 2:** Declare a variable 'Number' and set Default Value as 1

**Step 3:** While 'Number' < 100

**Step 4:** Number = Number + 2

**Step 5:** STOP

## Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘This is the code to generate the odd number series from 1 to 100’**.

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘// To generate odd number series from 1 to 100 by using while loop activity.**

**Step 6:** Drag and drop the **“Write Line”** activity from the activity panel and name it as **Write - ‘To display the text’** and inside the text field write **“The numbers from 1 to 100 are:”**

**Step 7:** Create a variable from the **“Variables”** panel in the main **“Sequence”** activity as under:

| Name   | Variable Type | Scope                                                                      | Default |
|--------|---------------|----------------------------------------------------------------------------|---------|
| Number | Int32         | Sequence – ‘This is the code to generate Even number series from 1 to 100’ | 1       |

**Step 8:** Drag and drop the **“While”** activity from the activity panel and name it as **While – ‘To check if the number is less than 100’**.

**Step 9:** Right-click on **“While”** activity select **Annotation > Add Annotation ‘To Print odd number series from 1 to 100. This block of code will execute until the number printed is 100’**.

**Step 10:** Inside the while activity write the condition **“Number < 100”**.

**Step 11:** In the body section of **“While”** activity drag and drop the **“Sequence”** activity.

**Step 12:** Change the **“Sequence”** activity name **Sequence – ‘Increase number by 2 and display the odd number series’**.

**Step 13:** Right-click on **“Sequence”** activity select **Annotation > Add Annotation ‘To Increase number by 2 and print the odd numbers in the output panel’**.

**Step 14:** Drag and drop the **“Write line”** activity from the activity panel and name it as **Write – ‘Print ODD numbers from 1 to 100’** and inside the text field write **Number.ToString**.

**Step 15:** Drag and drop the **“Assign”** activity inside the sequence activity.

**Step 16:** Change the “Assign” activity name **Assign – ‘Add two to the previous value.’**

**Step 17:** Declare the ‘**Number**’ variable in the **TO** box and ‘**Number+2**’ in the value box.

## 2.3 Even number series

Objective: To code a Robot in UiPath Studio to generate a series of even numbers from 2 to 100

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Write Line” activity and how to set **variables** in a code by using “Variable” panel.
- ✓ “While” activity and how to set conditions.
- ✓ Display output in “Output” panel.

### Algorithm:

**Step 1:** START

**Step 2:** Declare a variable 'Number' and set Default Value as 0

**Step 3:** While 'Number' < 100

**Step 4:** Number = Number + 2

**Step 5:** STOP

## Step by Step process:

**Step 1:** Open UiPath Studio

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘This is the code to generate Even number series from 2 to 100’**.

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘// To generate Even number series from 2 to 100 by using While Loop activity.**

**Step 6:** Drag and drop the **“Write line”** activity from the activity panel and name it as **Write line – ‘To display the text’** and inside the text field write **“The Even numbers from 2 to 100 are:”**

**Step 7:** Create a variable from the **“Variables”** panel in the main sequence activity as under:

| Name   | Variable Type | Scope                                                                      | Default |
|--------|---------------|----------------------------------------------------------------------------|---------|
| Number | Int32         | Sequence – ‘This is the code to generate Even number series from 1 to 100’ | 0       |

**Step 8:** Drag and drop the **“While”** activity from the activity panel and name it as **While – ‘To check if the number is less than 100’**.

**Step 9:** Right-click on **“While”** activity select **Annotation > Add Annotation ‘To print the even number series from 2 to 100. This block of code will execute until the number printed is 100’**.

**Step 10:** Inside the **“While”** activity write the condition **“Number < 100”**.

**Step 11:** In the body section of **“While”** activity drag and drop the **“Sequence”** activity in the body section.

**Step 12:** Change the **“Sequence”** activity name **Sequence – ‘Increase number by 2 and display the number series’**.

**Step 13:** Right-click on **“Sequence”** activity select **Annotation > Add Annotation ‘To Increase number by 2 and print the numbers in the output panel’**.

**Step 14:** Drag and drop the **“Assign”** activity inside the **“Sequence”** activity.

**Step 15:** Change the “Assign” activity name **Assign – ‘Add one to the previous value.’**

**Step 16:** Declare the ‘Number’ variable in the **TO** box and ‘Number+2’ in the value box.

**Step 17:** Drag and drop the “Write line” activity from the activity panel and name it as **Write Line – ‘Print numbers from 1 to 100’** and inside the text field write **Number.ToString**.

## 2.4 Prime number series

Objective: To code a Robot in UiPath Studio to generate a series of prime numbers from 1 to infinite.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Write Line” activity and how to set **variables** in a code by using “Variable” panel.
- ✓ “While” activity and how to set conditions.
- ✓ Display output in “Output” panel.

### Algorithm:

**Step 1:** START

**Step 2:** Declare the variables as 'Starting\_Index' , 'Number' , 'PrimeNumber' , 'Last\_Index'

**Step 3:** Take input from the user highest number to be input put condition as 'Starting\_Index < Last\_Index' assign 'Number' = 'Starting\_Index'

**Step 4:** Check 'Number >= 1' and then assign 'Starting\_Index mod Number = 0'

**Step 5:** Increment counter by 1 and decrement Number by 1

**Step 6:** Add condition as 'Counter = 2', Assign 'PrimeNumber' = 'PrimeNumber + Starting\_Index.ToString + " "'

**Step 7:** Increment Starting\_Index by 1 then print the series

**Step 8:** STOP

## Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence - 'This is a code to generate prime number series from 1 to infinite'**

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘//User will tell bot till which number he wants to get the prime numbers, then bot will print the prime number series from 0 to user input number.**

**Step: 6:** Drag and drop the **“Input Dialog”** activity from the activity panel.

**Step 7:** Name it as **Input Dialog - 'Taking input from the user in Last\_Index variable.'** and write the values as under:

| Title       | Label                                                                             |
|-------------|-----------------------------------------------------------------------------------|
| Enter Value | "Please enter the highest number upto which you want to print the prime numbers". |

**Step 8:** Create a variable through **“Variables”** panel for the **Input Dialog - 'Taking input from the user in Last\_Index variable.'** activity as under:

| Name       | Variable Type | Scope                                                                                 | Default |
|------------|---------------|---------------------------------------------------------------------------------------|---------|
| Last_Index | Int32         | Sequence – ‘This is a code to generate a series of prime numbers from 1 to infinite.’ |         |

**Step 9:** Declare the variable **Last\_Index** Value in the **“Output”** Property of **Input Dialog - 'Taking input from the user in Last\_Index variable.'**

**Step 10:** Drag and drop the **“Assign”** activity from the activity panel.

**Step 11:** Name it as **Assign- 'Take Starting\_Index as 1.'**

**Step 12:** Create a variable through **“Variables”** panel for the **Assign- 'Take Starting\_Index as 1.'** activity as under:

| Name           | Variable Type | Scope                                                                                 | Default |
|----------------|---------------|---------------------------------------------------------------------------------------|---------|
| Starting_Index | Int32         | Sequence - 'This is a code to generate a series of prime numbers from 1 to infinite.' | 0       |

**Step 13:** Declare the variable **Starting\_Index** in the “To” Property and 1 in “Value” Property of “Assign” activity.

**Step 14:** Drag and drop the “While” activity from the activity panel and name it as **While - 'Iterating the numbers until starting index is less than 100.'**

**Step 15:** Right-click on “While” activity select **Annotation > Add Annotation - 'Iterate till starting index value is less than the last index value and finding the prime numbers series between start\_Index to Last\_Index.'**

**Step 16:** Inside the “While” activity write the condition “**Starting\_Index<Last\_Index**”.

**Step 17:** In the body section of “While” activity drag and drop the “Sequence” activity.

**Step 18:** Name as the “Sequence” activity as **Sequence - 'To Find the Prime number series'.**

**Step 19:** Drag and drop the “Assign” activity from the activity panel.

**Step 20:** Name the “Assign” activity as **Assign - 'Pass the variable value Starting\_Index value to Number.'**

**Step 21:** Create a variable through “Variables” panel for the **Assign - 'Pass the variable value Starting\_Index value to Number.'** activity as under:

| Name   | Variable Type | Scope                                                                                 | Default |
|--------|---------------|---------------------------------------------------------------------------------------|---------|
| Number | Int32         | Sequence - 'This is a code to generate a series of prime numbers from 1 to infinite.' | 0       |

**Step 22:** Declare the variable **Number** in the “To” Property and use the **Starting\_Index** variable in “Value” Property of “Assign” activity.

**Step 23:** Drag and drop the “While” activity from the activity panel inside the “While” activity and name it as **While - 'Iterating till Number is greater than or equal to 1.'**

**Step 24:** Right-click on **“While”** activity select **Annotation > Add Annotation – ‘To check if the number is divisible by any other number or not, means number is prime or not.’**

**Step 25:** Inside the **“While”** activity write the **condition “Number>=1”**.

**Step 26:** In the body section of **“While”** activity drag and drop the **“Sequence”** activity.

**Step 27:** Name the **“Sequence”** activity as **Sequence - 'Decrease the number value by 1 and Increment the counter value by 1 if Starting\_Index mod Number is 0.'**

**Step 28:** Drag and drop the **“IF”** activity from the activity panel and name it as **IF - 'Increment the counter value by 1 if Starting\_Index mod Number is 0.'**

**Step 29:** Use the **Starting\_index mod Number = 0** Variable in the condition box of **“IF”** activity.

**Step 30:** Drag and drop the **“Assign”** activity in the **‘Then’** box of If activity.

**Step 31:** Name it as **“Assign”** activity as **Assign - 'Counter by increment by 1.'**

**Step 32:** Create a variable through **“Variables”** panel for the **Assign - 'Counter by increment by 1.'** activity as under:

| Name    | Variable Type | Scope                                                                                                            | Default |
|---------|---------------|------------------------------------------------------------------------------------------------------------------|---------|
| Counter | Int32         | Sequence – Decrease the number value by 1 and Increment the counter value by 1 if Starting_Index mod Number is 0 | 0       |

**Step 33:** Declare the variable **Counter** in the **“To”** Property and **Counter+1** in **“Value”** Property of **“Assign”** activity.

**Step 34:** Drag the **“Assign”** activity from the activity panel and drop it outside the **“IF”** activity.

**Step 35:** Name it as **“Assign”** activity as **Assign - 'Number by decrement by -1.'**

**Step 36:** Declare the variable **Number+1** in the **“To”** Property and **Number-1** in **“Value”** Property of **“Assign”** activity.

**Step 36:** Drag the **“IF”** activity and name it as: **IF - 'Counter value is 2 then assign PrimeNumber and starting Index to Prime Number.'**

**Step 37:** Use the **‘Counter = 2’** variable in the condition box of **IF** activity.

**Step 38:** Drag and drop the **“Assign”** activity in the **‘Then’** box of If activity.

**Step 39:** Name it as “**Assign**” activity as **Assign - 'PrimeNumber and starting Index to Prime Number.'**

**Step 40:** Create a variable through “**Variables**” panel for the **Assign - 'PrimeNumber and starting Index to Prime Number.'** activity as under:

| Name        | Variable Type | Scope                                                     | Default |
|-------------|---------------|-----------------------------------------------------------|---------|
| PrimeNumber | String        | Sequence – PrimeNumber and starting Index to Prime Number | ""      |

**Step 41:** Declare the variable **PrimeNumber** in the “**To**” Property and **PrimeNumber + Starting\_Index.ToString + " "** in “**Value**” Property of “**Assign**” activity.

**Step 42:** Drag the “**Assign**” activity from the activity panel and drop it outside the “**IF**” activity.

**Step 43:** Name it as “**Assign**” activity as **Assign - 'Starting\_Index by increment by 1.'**

**Step 44:** Use the variable **Starting\_Index** in the “**To**” Property and **Starting\_Index+1** in “**Value**” Property of “**Assign**” activity.

**Step 45:** Drag the “**Write Line**” activity from the activity panel and drop it in the main sequence activity.

**Step 46:** Name it as **Write Line - 'Print the Prime number series.'**

**Step 47:** Inside the text property write the text "**Prime Number series from 1 to**  
"+**Last\_Index.ToString**+" are :- " +**Environment.NewLine** +  
" " +**PrimeNumber**

## 2.5 Number order sorting

**Objective:** To code a Robot in UiPath Studio to sort numbers in Ascending and Descending order in an excel file.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Excel Application Scope” activity.
- ✓ “Read Range” and “Write Range” activity.
- ✓ “Sort Data Table” activity.

### Algorithm:

**Step 1:** START

**Step 2:** Declare variables as 'DataTable1' , 'DataTable2'

**Step 3:** Add Excel Application scope activity to read the excel file

**Step 4:** Add Read Range Activity to Read the Sheets

**Step 5:** Add Sort Data Table activity and change order as Ascending

**Step 6:** Add Sort Data Table activity and change order as Descending

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘This is a code to sort the numbers in ascending and descending order in an excel file.’**

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘// To sort the numbers in ascending and descending order in an excel file using Sort Data Table activity.’**

**Step 6:** Drag and drop the **“Excel Application Scope”** activity from the activity panel and name it as **Excel Application Scope - ‘This activity is used for performing any action on an excel sheet.’**

**Step 7:** In the **“Excel Application Scope”** activity text field specify the name of the excel file to be used in the code. In this code, the name of the file used is **“With\_DataTable.xlsx,”** so we will write this name in the text field.

**Step 8:** Change the name of **“Do”** sequence from the **“Excel Application Scope”** activity as **Do – ‘To read, write, and sort the data in the excel file’.**

**Step 9:** Right-click on the **“Do”** sequence and select **Annotation > Add Annotation ‘Read, write and sort the data in ascending and descending order by using Sort Data Table activity.’**

**Step 10:** Select the **ExcelReadRange** activity from the activities panel and drop in the sequence activity workflow.

**Step 11:** Change the name of **“ExcelReadRange”** activity as **ExcelReadRange – ‘This activity is used to read an excel sheet’** and write the name of excel sheet as mentioned in the excel file used. In this case, the sheet name used is **‘Sheet 1’**.

**Step 12:** Create two variables from the **“Variables”** panel in the **“ExcelReadRange”** activity as under:

| Name       | Variable Type | Scope                                                      | Default |
|------------|---------------|------------------------------------------------------------|---------|
| DataTable1 | DataTable     | Do – ‘To read, write, and sort the data in the excel file’ |         |
| DataTable2 | DataTable     | Do – ‘To read, write, and sort the data in the excel file’ |         |

**Step 13:** Declare the variable **DataTable1** in the “**Output**” Property of “**Read Range**” activity.

**Step 14:** Drag and drop the “**SortData Table**” activity and change the name it as – **SortData Table - ‘This activity will sort the data of the excel sheet in ascending order’**.

**Step 15:** To sort the data in ascending order, write the following values in the **Properties** panel of **SortData Table** activity as mentioned below.

| <b>Input (Data Table)</b> | <b>Output (Data Table)</b> | <b>Name</b> | <b>Order</b> |
|---------------------------|----------------------------|-------------|--------------|
| DataTable1                | DataTable2                 | “S.no”      | Ascending    |

**Note:**

- **DataTable1** declare in the “**Input Property**” of “**SortData Table**” activity.
- **DataTable2** declare in the “**Output Variable**” of “**SortData Table**” activity.
- **S.no** declare for the name of sorting column in the “**SortData Table**” activity

**Step 16:** Select **ExcelWriteRange** activity from the activities panel and drop it in the sequence workflow. Rename it as **ExcelWriteRange - ‘Write the sorted data in the specified column of excel sheet’**.

**Step 17:** In the “**ExcelWriteRange**” property, write the following values in **ExcelWriteRange** properties panel:

| <i>Sheet Name</i> | <i>Cell</i> | <i>Data Table</i> |
|-------------------|-------------|-------------------|
| “Sheet1”          | “B2”        | DataTable2        |

**Note:**

- **Sheet1** declare in the “**Sheet name**” of the **ExcelWriteRange** activity.
- **B2** declare in the “**StartingCell**” of the **ExcelWriteRange** activity
- **DataTable2** declare in the “**Input Property**” of “**Write Range activity**”

**Step 18:** Drag and drop the “**SortData Table**” activity and name it as – **SortData Table - ‘This activity will sort the data of the excel sheet in descending order’**.

**Step 19:** To sort the data in descending order, write the following values in the **Properties** of **SortData Table** activity as mentioned below.

| Input (Data Table) | Output (Data Table) | Name   | Order      |
|--------------------|---------------------|--------|------------|
| DataTable1         | DataTable2          | "S.no" | Descending |

**Note:**

- **DataTable1** declare in "Input Property" of "Sort DataTable activity".
- **DataTable2** declare in "Output Property" of "Sort DataTable" activity.
- **S.no** declare in the **Sorting column** of **SortData Table** activity.

**Step 20:** Select **ExcelWriteRange** activity from the activities panel and drop it in the sequence workflow. Change the name it as **ExcelWriteRange - 'Write the sorted data in the specified column of excel sheet'**.

**Step 21:** In the "ExcelWriteRange" activity properties panel, write the following values in the **ExcelWriteRange** property panel as mentioned values:

| <i>Sheet Name</i> | <i>Cell</i> | <i>Data Table</i> |
|-------------------|-------------|-------------------|
| "Sheet1"          | "C2"        | DataTable2        |

**Note:**

- **Sheet1** declare in the "Sheet name" of the **ExcelWriteRange** activity.
- **C2** declare in the "StartingCell" of the **ExcelWriteRange** activity.

**DataTable2** declare in the "Input Property" of "ExcelWriteRange" activity.

## 3 Variable swapping

### 3.1 Using three bucket method

**Objective:** To code a Robot in UiPath Studio to "swap two numbers" by using three bucket method.

#### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Input Dialog” activity.
- ✓ “Write Line” activity.

#### Algorithm:

**Step 1:** START

**Step 2:** Declare a variable 'First\_Input\_Value' , 'Second\_Input\_Value' , 'Swapping\_Support\_Variable'

**Step 3:** Swapping\_Support\_Variable = First\_Input\_Value

First\_Input\_Value = Second\_Input\_Value

Second\_Input\_Value = Swapping\_Support\_Variable

**Step 4:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘This code is for swapping two numbers by using a third variable’**.

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘// Taking input of two numbers from user and swap them by using a third variable’**.

**Step 6:** Drag the **“Sequence”** activity from the activity panel name it as **Sequence – ‘For prompting user to give input’**.

**Step 7:** Right click on **“Sequence”** activity, select **Annotation > Add Annotation ‘This block of code will prompt the user to give input’**.

**Step 8:** Drag and Drop **“Input Dialog”** activity and name it as **Input – ‘First Variable by User’** and write the values as under:

| Title       | Label                                   |
|-------------|-----------------------------------------|
| First Value | “Please enter the first numeric value:” |

**Step 9:** Create a variable through **“Variables”** panel for the **“Input – ‘First Variable by User’”** activity as under:

| Name              | Variable Type | Scope                                                                         | Default |
|-------------------|---------------|-------------------------------------------------------------------------------|---------|
| First_Input_Value | Double        | Sequence – ‘This code is for swapping two numbers by using a third variable’. |         |

**Step 10:** Declare the variable First\_Input\_Value in the **“Output”** Property of **“Input Dialog - ‘First variable by User’”**.

**Step 11:** Drag and Drop **“Input Dialog”** activity and name it as **Input – ‘Second variable by User’** and create the variables as under:

| Title        | Label                                    |
|--------------|------------------------------------------|
| Second Value | “Please enter the second numeric value:” |

**Step 12:** Create a variable through “**Variables**” panel for the “**Input – ‘Second variable by User’**” activity as under:

| Name               | Variable Type | Scope                                                                         | Default |
|--------------------|---------------|-------------------------------------------------------------------------------|---------|
| Second_Input_Value | Double        | Sequence – ‘This code is for swapping two numbers by using a third variable’. |         |

**Step 13:** Declare the variable **Second\_Input\_Value** in the “**Output**” Property of “**Input – ‘Second variable by User’**”.

**Step 14:** Drag and drop the “**Write line**” activity from the activity panel and name it as **Write line – ‘Value entered before swapping’** and declare the variable in text box as under:

**"First Value is : " + First\_Input\_Value.ToString + Environment.NewLine +**

**"Second Value is : "+ Second\_Input\_Value.ToString**

**Step 15:** Drag the “**Sequence**” activity from the activity panel and name it as **Sequence – ‘Swapping of numbers’**

**Step 16:** Right click on “**Sequence**” activity, select **Annotation > Add Annotation ‘This block of code will swap the values of the numbers entered’**.

**Step 17:** Create a variable through “**Variables**” panel for “**Assign**” activity as under:

| Name                      | Variable Type | Scope                                                                         | Default |
|---------------------------|---------------|-------------------------------------------------------------------------------|---------|
| Swapping_Support_Variable | Double        | Sequence – ‘This code is for swapping two numbers by using a third variable’. |         |

**Step 18:** Drag and Drop the “**Assign**” activity in the “**Sequence**” activity and name it as **Assign – ‘Move the First\_Input\_Value to Swapping\_Support\_Variable’**.

**Step 19:** Declare the variable **Swapping\_Support\_Variable** in the “**To**” Property and **First\_Input\_Value** in the “**Value**” Property of “**Assign**” activity.

**Step 20:** Drag and Drop the “**Assign**” activity in the “**Sequence**” activity and name it as **Assign – ‘Move the Second\_Input\_Value to First\_Input\_Value’**.

**Step 21:** Declare the variable **First\_Input\_Value** in the “**To**” Property and **Second\_Input\_Value** in the “**Value**” Property of “**Assign** activity.

**Step 22:** Drag and Drop the “**Assign**” activity in the “**Sequence**” activity and name it as **Assign – ‘To swap Swapping\_Support\_Variable with Second\_Input\_Value’**.

**Step 23:** Declare the variable **Second\_Input\_Value** in the “**To**” Property and **Swapping\_Support\_Value** in the “**Value**” Property of “**Assign** activity.

**Step 24:** Drag and drop “**Write Line**” activity and name it as **Write Line – ‘Swapped Result’** and write the following in the text box

**"First Value after swapping is : " + First\_Input\_Value.ToString  
+Environment.NewLine+"Second Value after swapping is : "+ Second\_Input\_Value.ToString".**

### 3.2 By using two variables only

**Objective:** To code a Robot in UiPath Studio to "Swap two numbers" by using two variables only.

#### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ "Sequence" and "Assign" activity.
- ✓ "Comment" and "Annotation".
- ✓ "Input Dialog" activity.
- ✓ "Write Line" activity.

#### Algorithm:

**Step 1:** START

**Step 2:** Declare a variable 'First\_Input\_Value' , 'Second\_Input\_Value'

**Step 3:** First\_Input\_Value = First\_Input\_Value + Second\_Input\_Value

Second\_Input\_Value = First\_Input\_Value - Second\_Input\_Value

First\_Input\_Value = First\_Input\_Value - Second\_Input\_Value

**Step 4:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘This code is for swapping two numbers without using a third variable’**.

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘// Taking input of two numbers from user and swap their values’**.

**Step 6:** Drag the **“Sequence”** activity from the activity panel name it as **Sequence – ‘For prompting user to give input’**.

**Step 7:** Right click on **“Sequence”** activity, select **Annotation > Add Annotation ‘This block of code will prompt the user to give input’**.

**Step 8:** Drag and Drop **“Input Dialog”** activity and name it as **Input – ‘First variable by user’** and write the values as under:

| Title       | Label                                   |
|-------------|-----------------------------------------|
| First Value | “Please enter the first numeric value:” |

**Step 9:** Create a variable through **“Variables”** panel for the **“Input – ‘First variable by user’”** activity as under:

| Name              | Variable Type | Scope                                                                              | Default |
|-------------------|---------------|------------------------------------------------------------------------------------|---------|
| First_Input_Value | Double        | Sequence – ‘This code is for swapping two numbers without using a third variable’. |         |

**Step 10:** Declare the variable **First\_Input\_Value** in the **“Output”** Property of **“Input – ‘First variable by user’”**.

**Step 11:** Drag and Drop **“Input Dialog”** activity and name it as **Input – ‘Second variable by user’** and create the variables as under:

| Title        | Label                                    |
|--------------|------------------------------------------|
| Second Value | “Please enter the second numeric value:” |

**Step 12:** Create a variable through “**Variables**” panel for the “**Input – ‘Second variable by user’**” activity as under:

| Name               | Variable Type | Scope                                                                              | Default |
|--------------------|---------------|------------------------------------------------------------------------------------|---------|
| Second_Input_Value | Double        | Sequence – ‘This code is for swapping two numbers without using a third variable’. |         |

**Step 13:** Declare the variable **Second\_Input\_Value** in the “**Output**” Property of “**Input – ‘Second variable by user’**”.

**Step 14:** Drag and drop the “**Write line**” activity from the activity panel and name it as **Write line – ‘Value entered before swapping’** and declare the variable in text box as under:

**"First Value is : " + First\_Input\_Value.ToString + Environment.NewLine +**

**"Second Value is : " + Second\_Input\_Value.ToString**

**Step 15:** Drag the “**Sequence**” activity from the activity panel and name it as **Sequence – ‘Swapping of numbers’**.

**Step 16:** Right click on “**Sequence**” activity, select **Annotation > Add Annotation ‘This block of code will swap the values of the numbers entered’**.

**Step 17:** Drag and Drop the “**Assign**” activity in the “**Sequence**” activity and name it as **Assign – ‘Add First\_Input\_Value with Second\_Input\_Value and store the result in First\_Input\_Value’**.

**Step 18:** Declare the variable **First\_Input\_Value** in the “**To**” Property and **First\_Input\_Value + Second\_Input\_Value** in “**Value**” Property of “**Assign**” activity.

**Step 19:** Drag and Drop the “**Assign**” activity in the “**Sequence**” activity and name it as **Assign – ‘Subtract Second\_Input\_Value from First\_Input\_Value and store the result in Second\_Input\_Value’**.

**Step 20:** Declare the variable **Second\_Input\_Value** in the “**To**” Property and **First\_Input\_Value - Second\_Input\_Value** in “**Value**” Property of “**Assign**” activity.

**Step 21:** Drag and Drop the “**Assign**” activity in the “**Sequence**” activity and name it as **Assign – ‘Subtract the Second\_Input\_Value from First\_Input\_Value’**.

**Step 22:** Declare the variable **First\_Input\_Value** in the “**To**” Property and **Fist\_Input\_Value - Second\_Input\_Value** in “**Value**” Property of “**Assign**” activity.

**Step 23:** Drag and drop **“Write Line”** activity and name it as **Write Line – ‘Swapped Result’** and write the following in the text box

**"First Value after swapping is : " + First\_Input\_Value.ToString  
+Environment.NewLine+"Second Value after swapping is : "+ Second\_Input\_Value.ToString".**

## 4 Print “Hello”

### 4.1 Print "Hello" by using Sequence activity

Objective: To code a Robot in UiPath Studio to print "Hello" in a message box and write line by using “Sequence” activity.

#### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Message Box” activity.
- ✓ “Write Line” activity.

#### Packag

#### Algorithm:

**Step 1:** START

**Step 2:** Add Message Box activity and write " Hello" into Flow Chart Activity

**Step 3:** Add Write Line activity and write "Hello" in Flow Chart Activity

**Step 4:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence - ‘This code is an example of Sequence activity to display Hello using message box and write line activities’**.

**Step 5:** Drag and Drop the **“Comment”** activity in the **“Sequence”** activity and write the comment **‘// Write Hello using message box & write line activities’**.

**Step 6:** Drag and Drop the Message Box in **“Sequence”** activity and name it as **Message Box - ‘To display Hello in a message box pop up’**.

**Step 7:** In the **Text** property of **“Message Box - ‘To display Hello in a message box pop up’”** write **“hello”**.

**Step 8:** Drag and drop **“Write line”** activity and name it as **Write Line - ‘To display Hello in the output panel’**.

**Step 9:** In the **Text** property of **“Write line - ‘To display Hello in the output panel’”** write **“hello”**.

## 4.2 Print "Hello" by using Flowchart activity

**Objective:** To code a Robot using UiPath Studio Print "Hello" display "Hello" in a message box and write line by using **"Flowchart"** activity.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ **"Flowchart"** activity.
- ✓ **"Comment"** and **"Annotation"**.
- ✓ **"Message Box"** activity.
- ✓ **"Write Line"** activity.

### Algorithm:

**Step 1:** START

**Step 2:** Add Message Box activity and write " Hello" in **"Sequence"** activity

**Step 3:** Add Write Line activity and write "Hello" in **"Sequence"** activity

**Step 4:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Flowchart”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Flowchart”** activity as **Flowchart - ‘This code is an example of Flowchart activity to display Hello using message box and write line activities’**.

**Step 5:** Drag and Drop the **“Comment”** activity in the **“Flowchart”** activity connect it with the **Start node** and write the comment **‘// Write Hello using message box & write line activities’**.

**Step 6:** Drag and Drop the Message Box in **“Flowchart”** activity connect it with **“Comment”** activity and name it as **Message Box – ‘To display Hello in a message box pop up’**.

**Step 7:** In the **Text** property of **“Message Box – ‘To display Hello in a message box pop up’”** write **“hello”**.

**Step 8:** Drag and drop **“Write line”** activity connect it with **“Message Box – ‘To display Hello in a message box pop up’** and name it as **Write Line – ‘To display Hello in the output panel’**.

## 5 Addition of two numbers

**Objective:** Code a Robot in UiPath Studio to display the sum of two numbers by taking them as input and display the output in a message box, until one of the input is 0.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Do While” activity and how to set conditions.
- ✓ “Input Dialog” activity and how to set **variables** in a code using “Variable” panel.
- ✓ Display output in “Message Box”.

### Algorithm:

**Step 1:** START

**Step 2:** Declare a variable 'FirstNumber' , 'Sum' , 'SecondNumber'

**Step 3:** Sum = FirstNumber + SecondNumber

**Step 4:** Do While FirstNumber < > 0 AND SecondNumber < > 0

**Step 5:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence - 'This is a code to display the sum of two numbers until one of the input entered is 0'**.

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘// Code to display the sum of two number by taking them as input from user and display the output until one of the input is 0'**.

**Step 6:** Drag and drop the **“Do While”** activity from the activity panel and name it as **Do While - 'Addition of numbers until one of the input value is not 0'**.

**Step 7:** Right-click on **“Do While”** activity, select **Annotation > Add Annotation 'To take input from user and display the sum of numbers. This code will execute until one of the input is 0'**.

**Step 8:** In the body section of **“Do While”** activity drag and drop the **“Sequence”** activity.

**Step 9:** Change the **“Sequence”** activity name **Sequence - ‘To add two numbers and display the output in a message box’**.

**Step 10:** Create three variables from the **“Variables”** panel in the **“Sequence”** activity as under:

| Name         | Variable Type | Scope                                                                   | Default |
|--------------|---------------|-------------------------------------------------------------------------|---------|
| Sum          | Double        | Do While - "Addition of numbers until one of the input value is not 0'. |         |
| FirstNumber  | Double        | Do While - "Addition of numbers until one of the input value is not 0'. |         |
| SecondNumber | Double        | Do While - "Addition of numbers until one of the input value is not 0'. |         |

**Step 11:** Drag and drop the **“Input Dialog”** activity inside the **“Sequence”** activity and name it as **Input Dialog - 'First input by user'**.

**Step 12:** In the **“Input Dialog”** property, write the following values in **Input Dialog** properties panel:

| <i>Label</i>             | <i>Title</i>   | <i>Result</i> |
|--------------------------|----------------|---------------|
| "Enter the first number" | "First Number" | FirstNumber   |

**Step 13:** Drag and drop “**Input Dialog**” activity inside the “**Sequence**” activity and name it as **Input Dialog – 'Second input by user'**.

**Step 14:** In the “**Input Dialog**” property, write the following values in “**Input Dialog**” properties panel:

| <i>Label</i>              | <i>Title</i>    | <i>Result</i> |
|---------------------------|-----------------|---------------|
| "Enter the Second number" | "Second Number" | SecondNumber  |

**Step 15:** Drag and drop the “**Assign**” activity below the “**Input Dialog**” activity.

**Step 16:** Change the “**Assign**” activity and name it as **Assign – 'Addition of numbers entered by user and assign the value to the variable Sum'**.

**Step 17:** Declare the ‘**Sum**’ variable in the **To** section and ‘**FirstNumber + SecondNumber**’ in the **Value** section of Property.

**Step 18:** Drag and drop the “**Message Box**” activity from the activity panel and name it as **Message Box – 'Display the output'** and inside the text field write “**Sum of numbers entered is :-**” +Sum.ToString

**Step 19:** Inside the “**Do While**” activity, write the condition “**FirstNumber <>0 AND SecondNumber <>0**”.

## 6 Displaying a Sun Sign

**Objective:** Code a Robot in UiPath Studio to display the sun sign of an individual by entering the Date of birth of an individual and produce the output in the message box.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Open Browser” and “Maximize Window” activity.
- ✓ “If” activity and how to set conditions.
- ✓ “Input Dialog”, “Type Into”, and “Click” activity and how to set **variables** in a code using “Variable” panel.
- ✓ Display output in “Message Box”.

### Algorithm:

**Step 1:** START

**Step 2:** Open the URL using Open Browser Activity

**Step 3:** Declare the variables as 'Dates', 'Months', 'Years', 'SunSign', 'ElementExist'

**Step 4:** Use ElementExist variable to check text boxes are available in the browser

**Step 5:** Add If-Else block activity and add ElementExist in the condition text box

**Step 6:** In the If block use variables 'Dates', 'Months', 'Years' to store them

**Step 7:** In the Else block write "Sunshine couldn't be generated, Page not Found"

**Step 8:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘This code is for identifying the Sun Sign of an individual by entering Date of Birth.’**

**Step 4:** Add a **“Comment”** activity from the activity panel and write **‘// To generate Sun Sign of an individual by getting the date of birth as an input.’**

**Step 5:** Drag and drop the **“Open Browser”** activity from the activity panel and name it as **Open Browser – ‘Opens a specific browser using specified URL’** and inside the URL text field write:

**<https://astrologyfutureeye.com/astro-calculators/sun-sign-calculator>**

**Step 6:** Change the name of **“Do”** sequence from the **“Open Browser”** activity as **Do – ‘Get date of birth as input from user, entering it in the browser text boxes and display the sun sign.’**

**Step 7:** Drag the **“Maximize Window”** activity from the activity panel and drop it in the workflow.

**Step 8:** Name the **“Maximize Window”** activity as **Maximize Window – ‘To maximize the browser window.’**

**Step 9:** Drag and Drop **“Input Dialog”** activity and name it as **Input Dialog – ‘Date of birth entered by user’** and write the values as under:

| Title           | Label                                   |
|-----------------|-----------------------------------------|
| “Date of Birth” | "Enter Date in numbers like -1, 2 etc." |

**Step 10:** Create a variable through **“Variables”** panel for the **“Input Dialog”** activity as under:

| Name  | Variable Type | Scope                                                                                      | Default |
|-------|---------------|--------------------------------------------------------------------------------------------|---------|
| Dates | String        | Do – ‘Getting user input, entering it to the browser text boxes, and generating sun sign.’ |         |

**Step 11:** Declare the variable **Dates** in the **“Output”** Property of **“Input Dialog”** activity.

**Step 12:** Drag and Drop **“Input Dialog”** activity and name it as **Input Dialog – ‘Month entered by user’** and write the values as under:

| Title            | Label                      |
|------------------|----------------------------|
| “Month of Birth” | "Enter Month e.g., April." |

**Step 13:** Create a variable through “**Variables**” panel for the “**Input Dialog**” activity as under:

| Name   | Variable Type | Scope                                                                                      | Default |
|--------|---------------|--------------------------------------------------------------------------------------------|---------|
| Months | String        | Do – ‘Getting user input, entering it to the browser text boxes, and generating sun sign.’ |         |

**Step 14:** Declare the variable **Months** in the “**Output**” Property of “**Input Dialog**” activity.

**Step 15:** Drag and Drop “**Input Dialog**” activity and name it as **Input Dialog – ‘Year entered by user’** and write the values as under:

| Title           | Label                          |
|-----------------|--------------------------------|
| “Year of Birth” | "Enter year in YYYY eg, 1995." |

**Step 16:** Create a variable through “**Variables**” panel for the “**Input Dialog**” activity as under:

| Name  | Variable Type | Scope                                                                                      | Default |
|-------|---------------|--------------------------------------------------------------------------------------------|---------|
| Years | String        | Do – ‘Getting user input, entering it to the browser text boxes, and generating sun sign.’ |         |

**Step 17:** Declare the variable **Years** in the “**Output**” Property of “**Input Dialog**” activity.

**Step 18:** Drag the “**Element Exists**” activity from the activity panel and drop it in the workflow.

**Step 19:** Name the “**Element Exists**” activity as **Element Exists – ‘To check if the sun sign finder block exists in the website.’**

**Step 20:** Create a variable through “**Variables**” panel for the “**Element Exists**” activity as under:

| Name         | Variable Type | Scope                                                                                     | Default |
|--------------|---------------|-------------------------------------------------------------------------------------------|---------|
| ElementExist | Boolean       | Do – ‘Getting user input, entering it to the browser text boxes and generating sun sign.’ |         |

**Step 21:** Declare the variable **ElementExist** in the “**Output**” Property of “**Element Exists**” activity.

**Step 22:** Drag the “**IF**” activity from the activity panel and drop it in the workflow.

**Step 23:** Name the “**IF**” activity as **IF – ‘Condition to check if the Sun sign finder exists or no. If exists then the code will continue, and if not then display an error message.’**

**Step 24:** Use the **ElementExist** Variable in the condition box of “**IF**” activity.

**Step 25:** Drag the “**Sequence**” activity from the activity panel and drop it in the “**Then**” workflow.

**Step 26:** Name the “Sequence” activity as **Sequence – ‘Enter Date of Birth (Date, Month, Year).’**

**Step 27:** Drag the “Sequence” activity from the activity panel and drop it in the workflow.

**Step 28:** Name the “Sequence” activity as **Sequence – ‘Date.’**

**Step 29:** Right-click on “Sequence” activity select **Annotation > Add Annotation ‘Entering and selecting the Date by identifying if in the dropdown menu.’**

**Step 30:** Drag the “Click” activity from the activity panel and drop it in the “Sequence” activity.

**Step 31:** Name the “Click” activity as **Click - 'To click and select the date from the date block.'**

**Step 32:** Drag the “Send Hotkey” activity from the activity panel and drop it in the “Sequence” activity.

**Step 33:** Name the “Send Hotkey” activity as **Click - 'To make the dropdown date visible.'**

**Step 34:** In the “Send Hotkey” activity select the “enter” key from the “Key” dropdown menu.

**Step 35:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 36:** Name the “Type Into” activity as **Type into - 'To select the date box.'**

**Step 37:** Declare the variable “Dates” in the “Input” Property of “Type Into” activity.

**Step 38:** Drag the “Sequence” activity from the activity panel and drop it in the **Sequence – ‘Enter Date of Birth (Date, Month, Year)’** workflow.

**Step 39:** Name the “Sequence” activity as **Sequence – ‘Month.’**

**Step 40:** Right-click on “Sequence” activity select **Annotation > Add Annotation ‘Entering and selecting the month by identifying it from the dropdown menu.’**

**Step 41:** Drag the “Click” activity from the activity panel and drop it in the “Sequence” activity.

**Step 42:** Name the “Click” activity as **Click - 'To Click and select the month from the month block.'**

**Step 43:** Drag the “Send Hotkey” activity from the activity panel and drop it in the “Sequence”.

**Step 44:** Name the “Send Hotkey” activity as **Click - 'To make the dropdown month visible.'**

**Step 45:** In the “Send Hotkey” activity select the “enter” key from the “Key” dropdown menu.

**Step 46:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 47:** Name the “Type Into” activity as **Type into - 'To Select the month box.'**

**Step 48:** Declare the variable “Months” in the “Input” Property of “Type Into” activity.

**Step 49:** Drag the **“Sequence”** activity from the activity panel and drop it in the **Sequence – ‘Enter Date of Birth (Date, Month, Year)’** workflow.

**Step 50:** Name the **“Sequence”** activity as **Sequence – ‘Year’**.

**Step 51:** Right-click on **“Sequence”** activity select **Annotation > Add Annotation ‘Entering and selecting the year by identifying it from the dropdown menu.’**

**Step 52:** Drag the **“Click”** activity from the activity panel and drop it in the **“Sequence”** activity.

**Step 53:** Name the **“Click”** activity as **Click - ‘To Click and select the year from the year block.’**

**Step 54:** Drag the **“Send Hotkey”** activity from the activity panel and drop it in the **“Sequence”**.

**Step 55:** Name the **“Send Hotkey”** activity as **Click - ‘To make the dropdown Year visible.’**

**Step 56:** In the **“Send Hotkey”** activity select the **“enter”** key from the **“Key”** dropdown menu.

**Step 57:** Drag the **“Type Into”** activity from the activity panel and drop it in the **“Sequence”** activity.

**Step 58:** Name the **“Type Into”** activity as **Type into - ‘To Select the year box.’**

**Step 59:** Declare the variable **“Years”** in the **“Input”** Property of **“Type Into”** activity.

**Step 60:** Drag the **“Click”** activity from the activity panel and drop it in the **Sequence – ‘Enter Date of Birth (Date, Month, Year)’** workflow.

**Step 61:** Name the **“Click”** activity as **Click – ‘Click on the calculate button to show the zodiac dashboard web screen.’**

**Step 62:** Drag the **“Get Value”** activity from the activity panel and drop it in the **“Sequence”** workflow.

**Step 63:** Name the **“Get Value”** activity as **Get Value – ‘Get the Sun Sign.’**

**Step 64:** Declare the variable **“Sun sign”** in the **“Output”** Property of **“Get Value”** activity.

**Step 65:** Drag the **“Message Box”** activity from the activity panel and drop it in the **“Sequence”** workflow.

**Step 66:** Name the **“Message Box”** activity as **Message Box - ‘To print Zodiac sun sign on screen.’**

**Step 67:** Drag the **“Message Box”** activity from the activity panel and drop it in the **“Else”** workflow of **If – ‘Sun sign finder exists, enter values else print message’** activity.

**Step 68:** Declare the variable **“Sun Sign couldn't be generated, Page not Found”** in the **“Input”** Property of **“Message box”** activity.

## 7 Guessing game

**Objective:** Code a Robot in UiPath Studio for creating a 'Guessing Game' with the following conditions:

- I. Generate a random number and prompt the user to input a number.
- II. In case of a wrong input a message is displayed to the user stating '**Please enter a lesser/greater number**'
- III. The loop keeps on running until the input number equals the generated number.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Flow Chart” and “Assign” activity.
- ✓ “Write Line” and “Do While”.
- ✓ “Sequence” and “Input Dialog” activity.
- ✓ “If” activity and how to set conditions.
- ✓ Display output in “Message Box”.

### Algorithm:

**Step 1:** START

**Step 2:** Declare the variables as 'RandomNo' , 'GuessedNo'

**Step 3:** Add decision box to the FlowChart and put the condition as  $\text{GuessedNo} < \text{RandomNo}$  add the message box to the true block and make the loop to the input box

**Step 4:** Add other decision box to the false branch and put the condition as  $\text{GuessedNo} = \text{RandomNo}$

**Step 5:** In the true branch of the decision box add "Congratulations you Guessed Correct No."

**Step 6:** In the false branch of the decision box add ""You guessed a higher No. Guess again!!""

**Step 7:** STOP

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Flowchart”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Flowchart”** activity as **Flowchart - 'Guessing Game by using Do While.'**

**Step 5:** Right-click on **“Flowchart”** activity select **Annotation > Add Annotation ‘Guessing Game: Generating a random no. using Random function then Guessing a no. and comparing it with random no. until user enter a no. equals to random no. generated.’**

**Step 6:** Drag the **“Assign”** activity from the activity panel and drop it in the workflow.

**Step 7:** Name the **“Assign”** activity as **Assign - 'Using random function to generate random value.'**

**Step 8:** Create a variable from the **“Variables”** panel in the main sequence activity as under:

| Name     | Variable Type | Scope                                          | Default |
|----------|---------------|------------------------------------------------|---------|
| RandomNo | Int32         | Flowchart - 'Guessing Game by using Do While.' | 0       |

**Step 9:** Declare the **‘RandomNo’** variable in the **TO** box and **‘new Random().next(0, 100)’** in the value box.

**Step 10:** Drag and drop the **“Write Line”** activity from the activity panel and name it as **Write Line - 'To check the random no. in output'** and inside the text field write **‘RandomNo.ToString’**.

**Step 11:** Drag and drop the **“Input Dialog”** activity inside the **“Sequence”** activity and name it as **Input dialog - 'to get the input of guessed no.'**

**Step 12:** Drag and drop the **“Flow Decision”** activity from the activity panel.

**Step 13:** Name the **“Flow Decision”** activity as **Flow Decision 1.**

**Step 14:** Drag and Drop the **Message Box** in the **“True”** side of **“Flow Decision”** activity and Name the **“Message box”** activity as **Message Box – ‘to print lower value.’**

**Step 15:** Drag and Drop the **Flow Decision** in the **“False”** side of **“Flow Decision”** activity and Name the **“Flow Decision”** activity as **Flow Decision 2.**

**Step 16:** Drag and Drop the **Message Box** in the **“True”** side of **“Flow Decision 2”** activity and Name the **“Message box”** activity as **Message Box – ‘to show Numbers are Equal.’**

**Step 17:** Drag and Drop the **Message Box** in the **“False”** side of **“Flow Decision 2”** activity and Name the **“Message box”** activity as **Message Box – ‘to show Number entered is higher.’**

**Step 18:** Connect the **Message Box – ‘to show Number entered is higher.’** Node to **Input dialog - 'to get the input of guessed number.'**

**Step 19:** Connect the **Message Box – ‘to print lower value.’** node to **Input dialog - 'to get the input of guessed number.'**

**Step 20:** Connect the start node to “**Assign, Write Line, Input Dialog and Flow Decision**” Activity.

## 8 Compare two columns of a spreadsheet

**Objective:** To code a Robot in UiPath Studio to compare two columns in a spreadsheet and display the result in third column if it is a match or not.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Excel Application Scope” activity.
- ✓ “Read Range” and “Write Cell” activity.
- ✓ “For Each Row” activity.
- ✓ “If” activity and how to set conditions.

### Package Prerequisites:

- UiPath.Excel.Activities = 2.6.0

### Algorithm:

**Step 1** - START

**Step 2** – Read all the rows in Column 1 and Column 2 of Excel

**Step 3** – Declare a variable RowNumber

**Step 4** - If Column 1 = Column 2 then it is a match else it is not.

**Step 5** - RowNumber = RowNumber + 1

**Step 6** – STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence - 'This is a code to compare two columns in a spreadsheet'**.

**Step 5:** Add a **“Comment”** activity from the activity panel and write **‘//Code to read an Excel, compare two columns and then write in Result column if it is a match or not.’**

**Step 6:** Drag and drop the **“Excel Application Scope”** activity from the activity panel and name it as **Excel Application Scope - ‘This activity is used for performing any action on an excel sheet.’**

**Step 7:** In the **“Excel Application Scope”** activity text field specify the name of the excel file to be used in the code. In this code, the name of the file used is **"Compare Columns.xlsx"** so we will write this name in the text field.

**Step 8:** Change the name of **“Do”** sequence from the **“Excel Application Scope”** activity as **Do – ‘To read, compare and write the data in the excel file’**.

**Step 9:** Right-click on the **“Do”** sequence and select **Annotation >Add Annotation - ‘Read an excel file for each row, check if column 1 and column 2 are equal and write in result column if it is a match or not’**

**Step 10:** Select the **Excel Read Range** activity from the activities panel and drop in the sequence activity workflow.

**Step 11:** Change the name of **Excel "Read Range"** activity as **Read Range– ‘This activity is used to read an excel sheet’** and write the name of excel sheet as mentioned in the excel file used. In this case, the sheet name used is **‘Sheet 1’**.

**Step 12:** Create two variables from the **“Variables”** panel in the **Excel "Read Range"** activity as under:

| Name       | Variable Type | Scope                                                        | Default |
|------------|---------------|--------------------------------------------------------------|---------|
| DataTable1 | DataTable     | Do – ‘To read, compare and write the data in the excel file’ |         |

|           |       |                                                              |  |
|-----------|-------|--------------------------------------------------------------|--|
| RowNumber | Int32 | Do – 'To read, compare and write the data in the excel file' |  |
|-----------|-------|--------------------------------------------------------------|--|

**Step 13:** Declare the variable **DataTable** in the “**Output**” Property of “**Read Range**” activity.

**Step 14:** Drag and drop the “**For Each Row**” activity and change the name it as – **For Each Row - 'Compare both the rows in the DataTable till the last record'**

**Step 15:** Declare **Row** in **ForEach** and **DataTable** in **in** section.

**Step 16:** Change the name of “**Body**” sequence from the “**For Each Row**” activity as **Do – 'Compare the columns (row wise)'**.

**Step 17:** Drag the “**IF**” activity from the activity panel and drop it in the workflow.

**Step 18:** Name the “**IF**” activity as **IF – 'column 1 equals column 2 then it's a match else it is not a match'**.

**Step 19:** Right-click on the “**IF**” activity and select **Annotation >Add Annotation – If column 1 equals column 2 then write in each corresponding cell as a match, else not a match against each row.**

**Step 20:** Inside the “**IF**” activity write the condition “**cint(row(0)) = cint(row(1))**”.

**Step 21:** In the **Then** section of “**IF**” activity add a “**Write Cell**” Excel activity and change the name from “**Write Cell**” as **Write Cell - 'Write "Match" in each cell if condition is met'**.

**Step 22:** In the “**Write Cell**” activity properties panel, write the following values in the property panel as mentioned below:

| <i>Sheet Name</i> | <i>Range</i>           | <i>Value</i> |
|-------------------|------------------------|--------------|
| “Sheet1”          | “C”+RowNumber.ToString | “Match”      |

**Step 23:** In the **Else** section of “**IF**” activity add a “**Write Cell**” Excel activity and change the name from “**Write Cell**” as **Write Cell - 'Write "Not a Match" in each cell if condition is not met'**

**Step 24:** In the “**Write Cell**” activity properties panel, write the following values in the property panel as mentioned below:

| <i>Sheet Name</i> | <i>Range</i>           | <i>Value</i>  |
|-------------------|------------------------|---------------|
| "Sheet1"          | "C"+RowNumber.ToString | "Not a Match" |

**Step 25:** Drag and drop the **"Assign"** activity below the **"If"** activity.

**Step 26:** Change the **"Assign"** activity name **Assign - 'Increment the RowNumber variable by 1'**

**Step 27:** Declare the **'RowNumber'** variable in the **To** box and **'RowNumber+1'** in the value box.

## 9 Disk cleanup

**Objective:** Code a Robot in UiPath Studio using the 'Recorder' feature for automating disk clean-up process.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ **Sequence** and **Assign** activity.
- ✓ **Comment** and **Annotation**.
- ✓ **Web Recording** activity.
- ✓ **Attach window** activity.
- ✓ **Type Into**, and **Click** activity.

### Algorithm:

**Step 1:** START

**Step 2:** Add the hot key to run the command prompt

**Step 3:** Add **Attach Window Activity** in the sequence activity to select the drive and clean up

**Step 4:** Add another window activity in the new sequence activity to select the specific drive

**Step 5:** Add another window activity in the new sequence activity for the DiskClean Dialog

**Step 6:** Add another window activity in the new sequence activity for the deleting the temporary files

**Step 7:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – Sequence - 'This code is for Cleaning the disk drive C, Erasing temp files.'**

**Step 4:** Add a **“Comment”** activity from the activity panel and write **‘//To open "Cleanmgr" with a prompt of selecting C drive and deleting all temporary and unused files.’**

**Step 5:** Drag and drop **“Send Hotkey”** activity inside the **“Sequence”** activity and name it as **Send Hotkey - 'This will open run command using keystrokes'**. Tick **‘win’** checkbox and write **‘r’** under key section.

**Step 6:** Click on **“Recording”** feature and select **“Desktop”** Recorder.

**Step 7:** A prompt will appear click on **“Record”** button from it and select the **‘textbox’** of run command.

**Step 8:** A pop up will appear having a text box, write **‘cleanmgr’** on it. Tick **Empty Field** checkbox and press **Enter**, this will write the text in **run command**.

**Step 9:** Press the **ok** button of run command, this will open a pop up for selecting drive,

**Step 10:** By default, C drive is selected, click **ok**.

**Step 11:** This will open a prompt with already selected options of **temporary internet files, downloaded programs**, click **ok**

**Step 12:** Recording is done press **esc** key then click **save and exit**.

**Step 13:** A popup will appear asking – **‘Do you want to keep the recording’** click **yes**.

**Step 14:** We will get a full code inside our **“Desktop”** sequence name it as **Desktop - 'Record to open run and clean up c drive using command cleanmgr.'**

**Step 15:** Name **“Attach Window”** activity as **Attach Window – ‘It specifically attaches the already open run command window to work on it.’**

**Step 16:** Name **“Do”** Sequence as **Do – ‘Type in to the "run" window, select the drive and clean it.’**

**Step 17:** Name **“Type Into”** activity as **Type Into - 'Types Cleanmgr into the textbox of run command.'**

**Step 18:** Name **“Click”** activity as **Click - 'Click on the ok Button.'**

**Step 19:** Name “**Attach Window**” activity as **Attach Window - 'This Specifies drive selection Window'**

**Step 20:** Name “**Do**” Sequence as **Do - 'Select the drive c and press ok.'**

**Step 21:** Name “**Click**” activity as **Click - 'Click on the ok button.'**

**Step 22:** Name “**Attach Window**” activity as **Attach Window - 'This Specifies DiskClean Dialog Window.'**

**Step 23:** Name “**Do**” Sequence as **Do - 'Click ok to process the clean-up'**

**Step 24:** Name “**Click**” activity as **Click - 'Click on the ok Button.'**

**Step 25:** Name “**Attach Window**” activity as **Attach Window - 'Specifies a new window to click delete'**

**Step 26:** Name “**Do**” Sequence as **Do - 'Click on delete files button to complete the clean-up Process'**

**Step 27:** Name “**Click**” activity as **Click - 'Click on delete files button'**

## 10 Extracting data from a website

**Objective:** Code a Robot in UiPath Studio to scrape data from a website and store it in .CSV File.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Open Browser” activity.
- ✓ “Type Into” activity.
- ✓ “Browser Scope” activity.
- ✓ “Extract Data” activity.
- ✓ “Write CSV” activity.

### Package Prerequisites:

- UiPath.Excel.Activities = 2.6.0

### Algorithm:

**Step 1:** START

**Step 2:** Open the URL using Open Browser Activity

**Step 3:** Declare variables as 'CSVFile' , 'ExtractDataTable' , 'Search\_Item' , 'URL'

**Step 4:** Use the variables in the different activity blocks to search , find the given item

**Step 5:** Output the result in the write csv file activity

**Step 6:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the “**Sequence**” activity from the activity panel and drop it in the workflow.

**Step 4:** Name the “**Sequence**” activity as **Sequence – ‘This is the code to extract data from a website in a .CSV file.’**

**Step 5:** Add a “**Comment**” activity from the activity panel and write ‘// **To extract data from a website in a .CSV file.**’

**Step 6:** Drag the “**Open Browser**” activity from the activity panel and drop it in the workflow.

**Step 7:** Name the “**Open Browser**” activity as **Open Browser – ‘To open the specified URL in Internet Explorer.’**

**Step 8:** Create a variable from the “**Variables**” panel in the “**Sequence**” activity as under:

| Name | Variable Type | Scope                                                                        | Default                                          |
|------|---------------|------------------------------------------------------------------------------|--------------------------------------------------|
| URL  | String        | Sequence – ‘This is the code to extract data from a website in a .CSV file.’ | <a href="http://www.amazon.in">www.amazon.in</a> |

**Step 8:** Declare the variable **URL** in the “**Input**” Property of “**Open Browser**” activity.

**Step 9:** Drag the “**Sequence**” activity from the activity panel and drop it in the workflow.

**Step 10:** Name the “**Sequence**” activity as **Sequence – ‘This block of code will search the data of iPhone (specified product) and extract in a.CSV file.’**

**Step 11:** Drag the “**Type Into**” activity from the activity panel and drop it in the “**Sequence**” activity.

**Step 12:** Name the “**Type Into**” activity as **Type into – ‘Type iPhone (specified product) in the search bar of Amazon (specified website).’**

**Step 13:** Create a variable from the “**Variables**” panel in the “**Type Into**” activity as under:

| Name        | Variable Type | Scope                                                                                              | Default  |
|-------------|---------------|----------------------------------------------------------------------------------------------------|----------|
| Search_Item | String        | ‘This block of code will search the data of iPhone (specified product) and extract in a.CSV file.’ | “iPhone” |

**Step 14:** Declare the variable “**Search\_item**” in the “**Input**” Property of “**Type Into**” activity.

**Step 15:** Drag the “**Click**” activity from the activity panel and drop it in the “**Sequence**” activity.

**Step 16:** Name the “Click” activity as **Click – ‘To search and display the results by clicking the search button on the webpage.’**

**Step 17:** Drag the “Sequence” activity from the activity panel.

**Step 18:** Name the “Sequence” activity as **Sequence – ‘To extract the result data from the website.’**

**Step 19:** Drag the “Attach Browser” activity from the activity panel and drop it in the “Sequence” activity.

**Step 20:** Name the “Attach Browser” activity as **Attach Browser – ‘It will attach the browser which was opened using Open Browser activity.’**

**Step 22:** Drag the “Extract Data” activity from the activity panel.

**Step 23:** Name the “Extract Data” activity as **Extract Data – ‘It will extract the data fields of iPhone (specified product).’**

**Step 24:** Create a variable from the “Variables” panel in the “Extract data” activity as under:

| Name             | Variable Type | Scope                                                  | Default                   |
|------------------|---------------|--------------------------------------------------------|---------------------------|
| ExtractDataTable | DataTable     | Sequence – To extract the result data from the website | New System.Data.DataTable |

**Step 25:** Declare the variable **ExtractDataTable** in the “Output” Property of “Extract Data” activity.

**Step 26:** Drag the “Write CSV” activity from the activity panel.

**Step 27:** Name the “Write CSV” activity as **Write CSV - ‘This will extract the data in the .CSV format from the website.’**

**Step 28:** Create a variable from the “Variables” panel in the “Write CSV” activity as under:

| Name    | Variable Type | Scope                                                                                                         | Default          |
|---------|---------------|---------------------------------------------------------------------------------------------------------------|------------------|
| CSVFile | String        | Sequence - ‘This block of code will collect the information from the results and compile it into a.CSV file.’ | "AmazonData.csv" |

**Step 29:** Declare the variable **ExtractDataTable** in the “Input” Property and **CSVFile** variable in the “FilePath” Property of “Write CSV” activity.

## 11 Filling a webform from an excel sheet

**Objective:** Code a Robot in UiPath Studio to fill a Webform from the data extracted from an excel sheet.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Assign” activity.
- ✓ “Comment” and “Annotation”.
- ✓ “Open Browser” and “Maximize Window” activity.
- ✓ “Excel Application Scope” activity and how to set conditions.
- ✓ “For Each Row”, “Excel Read Range”, “Type Into,” “Click” and “Get Row Item” activity and how to set **variables** in a code using “Variable” panel.

### Package Prerequisites:

- UiPath.Excel.Activities = 2.6.0

### Algorithm:

**Step 1:** START

**Step 2:** Declare the variables as 'FirstName' , 'LastName' , 'Company' , 'Role', 'Address', 'Email' , 'PhoneNo'

**Step 3:** Add browser activity and put ""https://forms.gle/BnonGQCaaY8QGzk46"" this link inside the browser activity

**Step 4:** Add the Excel application scope activity and add the excel file in it and read the excel file

**Step 5:** Add For Each row activity to read the contents of the excel file

**Step 6:** Add Get Row activity to read the excel file in row format

**Step 7:** Add one more sequence and add Type Into activity in it to read 'FirstName' , 'LastName' , 'Company' , 'Role', 'Address', 'Email' , 'PhoneNo'

**Step 8:** Drag and drop click activity to submit the form

**Step 9:** Drag and drop one more click activity to submit the other form response

**Step 10:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the “**Sequence**” activity from the activity panel and drop it in the workflow.

**Step 4:** Name the “**Sequence**” activity as **Sequence - 'This code is to automatically fill a webform from data stored in an excel sheet.'**

**Step 4:** Add a “**Comment**” activity from the activity panel and write ‘// **Open Browser webform, Read Excel sheet row by row and then enter the data in the webform accordingly.**’

**Step 5:** Drag and drop the “**Open Browser**” activity from the activity panel and name it as **Open Browser – ‘Opens the specified URL in the specified web browser’** and inside the URL text field write:

["https://forms.gle/BnonGQCaaY8QGzk46"](https://forms.gle/BnonGQCaaY8QGzk46)

**Step 6:** Right-click on “**Open Browser**” activity select **Annotation > Add Annotation ‘Open the browser and maximize it.’**

**Step 7:** Name the “**Do**” sequence in “**Open Browser**” activity as **Do - 'Get information as per user input, enter it in the browser text boxes and display "Your record has been recorded".'**

**Step 8:** Drag the “**Maximize Window**” activity from the activity panel and drop it in the workflow.

**Step 9:** Name the “**Maximize Window**” activity as **Maximize Window – ‘To maximize the browser window.’**

**Step 10:** Drag and Drop “**Excel Application Scope**” activity and name it as **Excel Application Scope – ‘To work on the specified excel sheet.’**

**Step 11:** Insert the name of the excel sheet in **WorkbookPath** property of “**Excel Application Scope**” activity.

**Note:** In this code the name of excel file is: ‘**Challenge1.xlsx**’

**Step 12:** Name the “**Do**” sequence of “**Excel Application Scope**” activity as **Do- 'Read the excel sheet and extract the data row by row until the condition is fulfilled.'**

**Step 13:** Drag and Drop “**Excel Read Range**” activity and name it as **Read Range - 'To read the excel sheet.'**

**Step 14:** Declare the sheet Name “**Sheet1**” in the “**Input**” Property and in the Range property of “**Read Range**” activity insert “”(double quotes).

**Step 15:** Create a variable through “Variables” panel for the “For Read Range” activity as under:

| Name      | Variable Type | Scope                                                                                        | Default |
|-----------|---------------|----------------------------------------------------------------------------------------------|---------|
| DataTable | DataTable     | Do- 'Read the excel sheet and extract the data row by row until the condition is fulfilled.' |         |

**Step 16:** Drag and Drop “For Each Row” activity and name it as **For Each Row - 'To extract the data and enter it in the webform until the condition is fulfilled.'**

**Step 17:** Declare the variable **DataTable** in the “Input” Property of “For Each Row” activity.

**Step 19:** Name the “Body” sequence as **Body - 'Extracting excel data & entering it in the webform'.**

**Step 20:** Drag and Drop “Sequence” activity and name it as **Sequence - 'This block of code is to extract excel data row by row.'**

**Step 21:** Right-click on “Sequence” activity select **Annotation > Add Annotation 'Get the values of the excel sheet, row by row and store it in the variables.'**

**Step 22:** Drag and Drop “Get Row Item” activity inside the “Sequence” activity.

**Step 23:** Name it as **Get Row Item - 'To extract the value of specified row and column.'**

**Step 24:** Create a variable through “Variables” panel for the “Get Row Item” activity as under:

| Name      | Variable Type | Scope                                                                | Default |
|-----------|---------------|----------------------------------------------------------------------|---------|
| FirstName | GenericValue  | Sequence - 'This block of code is to extract excel data row by row.' |         |

**Step 25:** Declare the variable **FirstName** in the “Output” Property of “Get Row Item” activity.

**Step 26:** Declare the “**FirstName**” in the “Column Name” property of “Get Row Item” activity.

**Step 27:** Inside the **Data table row** box write the “row” of “Get Row Item” activity.

**Step 28:** Drag and Drop “Get Row Item” activity inside the “Sequence” activity.

**Step 29:** Name it as **Get Row Item - 'To extract the value of specified row and column.'**

**Step 30:** Create a variable through “Variables” panel for the “Get Row Item” activity as under:

| Name     | Variable Type | Scope                                                                | Default |
|----------|---------------|----------------------------------------------------------------------|---------|
| LastName | GenericValue  | Sequence - 'This block of code is to extract excel data row by row.' |         |

**Step 31:** Declare the variable **LastName** in the “**Output**” Property of “**Get Row Item**” activity.

**Step 32:** Declare the “**LastName**” in the “**Column Name**” property of “**Get Row Item**” activity.

**Step 33:** Inside the **Data table row** box write the “**row**” of “**Get Row Item**” activity.

**Step 34:** Drag and Drop “**Get Row Item**” activity inside the “**Sequence**” activity.

**Step 35:** Name it as **Get Row Item - 'To extract the value of specified row and column.'**

**Step 36:** Create a variable through “**Variables**” panel for the “**Get Row Item**” activity as under:

| Name    | Variable Type | Scope                                                                | Default |
|---------|---------------|----------------------------------------------------------------------|---------|
| Company | GenericValue  | Sequence - 'This block of code is to extract excel data row by row.' |         |

**Step 37:** Declare the variable **Company** in the “**Output**” Property of “**Get Row Item**” activity.

**Step 38:** Declare the “**Company Name**” in the “**Column Name**” property of “**Get Row Item**” activity.

**Step 39:** Inside the **Data table row** box write the “**row**” of “**Get Row Item**” activity.

**Step 40:** Drag and Drop “**Get Row Item**” activity inside the “**Sequence**” activity.

**Step 41:** Name it as **Get Row Item - 'To extract the value of specified row and column.'**

**Step 42:** Create a variable through “**Variables**” panel for the “**Get Row Item**” activity as under:

| Name | Variable Type | Scope                                                                | Default |
|------|---------------|----------------------------------------------------------------------|---------|
| Role | GenericValue  | Sequence - 'This block of code is to extract excel data row by row.' |         |

**Step 43:** Declare the variable **Role** in the “**Output**” Property of “**Get Row Item**” activity.

**Step 44:** Declare the “**Role in Company**” in the “**Column Name**” property of “**Get Row Item**” activity.

**Step 45:** Inside the **Data table row** box write the “**row**” of “**Get Row Item**” activity.

**Step 46:** Drag and Drop “**Get Row Item**” activity inside the “**Sequence**” activity.

**Step 47:** Name it as **Get Row Item - 'To extract the value of specified row and column.'**

**Step 48:** Create a variable through “**Variables**” panel for the “**Get Row Item**” activity as under:

| Name    | Variable Type | Scope                                                                | Default |
|---------|---------------|----------------------------------------------------------------------|---------|
| Address | GenericValue  | Sequence - 'This block of code is to extract excel data row by row.' |         |

**Step 49:** Declare the variable **Address** in the “**Output**” Property of “**Get Row Item**” activity.

**Step 50:** Declare the “**Address**” in the “**Column Name**” property of “**Get Row Item**” activity.

**Step 51:** Inside the **Data table row** box write the “**row**” of “**Get Row Item**” activity.

**Step 52:** Drag and Drop “**Get Row Item**” activity inside the “**Sequence**” activity.

**Step 53:** Name it as **Get Row Item - 'To extract the value of specified row and column.'**

**Step 54:** Create a variable through “**Variables**” panel for the “**Get Row Item**” activity as under:

| Name  | Variable Type | Scope                                                                | Default |
|-------|---------------|----------------------------------------------------------------------|---------|
| Email | GenericValue  | Sequence - 'This block of code is to extract excel data row by row.' |         |

**Step 55:** Declare the variable **Email** in the “**Output**” Property of “**Get Row Item**” activity.

**Step 55:** Declare the “**Email**” in the “**Column Name**” property of “**Get Row Item**” activity.

**Step 56:** Inside the **Data table row** box write the “**row**” of “**Get Row Item**” activity.

**Step 57:** Drag and Drop “**Get Row Item**” activity inside the “**Sequence**” activity.

**Step 58:** Name it as **Get Row Item - 'To extract the value of specified row and column.'**

**Step 59:** Create a variable through “**Variables**” panel for the “**Get Row Item**” activity as under:

| Name    | Variable Type | Scope                                                                | Default |
|---------|---------------|----------------------------------------------------------------------|---------|
| PhoneNo | GenericValue  | Sequence - 'This block of code is to extract excel data row by row.' |         |

**Step 60:** Declare the variable **PhoneNo** in the “**Output**” Property of “**Get Row Item**” activity.

**Step 61:** Declare the “**Phone Number**” in the “**Column Name**” property of “**Get Row Item**” activity.

**Note:** From step 21 to 71, we have been created and declared **7 variables for 7 “Get Row Items”**.

**Step 62:** Inside the **Data table row** box write the “**row**” of “**Get Row Item**” activity.

**Step 63:** Drag and Drop “**Sequence**” activity inside the “**Body**” section of “**For each row.**”

**Step 64:** Name it as **Sequence - 'This block of code will enter the extracted data in the webform.'**

**Step 65:** Right-click on “**Sequence**” activity select **Annotation > Add Annotation** ‘**Pick the variable and enter it in the required text box of webform.**’

**Step 66:** Drag the “**Type Into**” activity from the activity panel and drop it in the “**Sequence**” activity.

**Step 67:** Name the “Type Into” activity as **Type Into - 'To insert the First Name in the Text Box.'**

**Step 68:** Use the **FirstName** variable in the text box of “Type Into” activity.

**Step 69:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 70:** Name the “Type Into” activity as **Type Into - 'To insert the Last Name in the Text Box.'**

**Step 71:** Use the **LastName** variable in the text box of “Type Into” activity.

**Step 72:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 73:** Name the “Type Into” activity as **Type Into - 'To insert the Company Name in the Text Box.'**

**Step 74:** Use the **Company** variable in the text box of “Type Into” activity.

**Step 75:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 76:** Name the “Type Into” activity as **Type Into - 'To insert the role in the Text Box.'**

**Step 77:** Use the **Role** variable in the text box of “Type Into” activity.

**Step 78:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 79:** Name the “Type Into” activity as **Type Into - 'To insert the Address in the Text Box.'**

**Step 80:** Use the **Address** variable in the text box of “Type Into” activity.

**Step 81:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 82:** Name the “Type Into” activity as **Type Into - 'To insert the Email in the Text Box.'**

**Step 83:** Use the **Email** variable in the text box of “Type Into” activity.

**Step 84:** Drag the “Type Into” activity from the activity panel and drop it in the “Sequence” activity.

**Step 85:** Name the “Type Into” activity as **Type Into - 'To insert the Phone Number in the Text Box.'**

**Step 86:** Use the **PhoneNo** variable in the text box of “Type Into” activity.

**Step 87:** Drag and drop the “Click” activity.

**Step 88:** Name it as **Click – 'to click submit button to fill the details.'**

**Step 89:** Drag and drop the “Click” activity.

**Step 90:** Name it as **Click – 'to submit another response'**

## 12 Extracting data from an invoice image

**Objective:** Code a Robot in UiPath Studio to read a scanned image of an invoice and store the extracted data in a .CSV file.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” activity.
- ✓ “Send Hotkey” and “Type Into”.
- ✓ “Get OCR Text” activity.
- ✓ “Build Data Table Into” activity.
- ✓ “Add Data Row” activity.
- ✓ “Write CSV” activity.

**Note:** OCR works on the specific requirement in this project. There can be variations in the photo application in your system. In this code, you need to use OCR according to your photo view application environment because for OCR a fixed environment is required.

### Package Prerequisites:

- **UiPath.Excel.Activities = 2.6.2**

### Algorithm:

**Step 1:** START

**Step 2:** Declare the variables as 'InvoiceTotal' , 'SubTotal' , 'ResultDT' , 'EmailAddress' , 'Tax'

**Step 3:** Add the Sequence in the main area and add the send Hot key to the Sequence activity

**Step 4:** Add Type Into activity again and add the OCR to get total of the invoice image

**Step 5:** Add again OCR activity to get 'Email' , 'Sub Total' , 'Tax Details' from the Tax invoice image.

**Step 6:** Add Build Data Table for insert data to data table and add the row activity to it

**Step 7:** Write it to the CSV file activity

**Step 8:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the “**Sequence**” activity from the activity panel and drop it in the workflow.

**Step 4:** Name the “**Sequence**” activity as **Sequence** – ‘**This is a code to read a scanned image of an invoice and store extracted data in .CSV file.**’

**Step 5:** Drag the “**Send Hotkey**” activity from the activity panel and drop it in the workflow.

**Step 6:** Name the “**Send Hotkey**” activity as **Send hotkey** - ‘**To open run command by entering win + r keystroke.**’

**Step 7:** Use the “**r**” key string in the “**input**” property of “**Send Hotkey**” activity.

**Step 8:** Drag the “**Type Into**” activity from the activity panel and drop it in the workflow.

**Step 9:** Name the “**Type Into**” activity as **Type Into** – ‘**Type the invoice file directory along with image name and extension.**’

**Note:**

- Please use the path in the “**Type Into**” activity where the invoice image file is saved.
- Example: **C:\Users\Ankit\Exercise - 12 ScannedInvoice\Sample.Test.png**

**Step 10:** Drag the “**Send Hotkey**” activity from the activity panel and drop it in the workflow.

**Step 11:** Name the “**Send Hotkey**” activity as **Send hotkey** - ‘**To click on OK button to display the image.**’

**Step 12:** Use the “**enter**” key string in the “**input**” property of “**Send Hotkey**” activity.

**Step 13:** Drag the “**Get OCR Text**” activity from the activity panel and drop it in the workflow.

**Step 14:** Name the “**Get OCR Text**” activity as **Get OCR Text** – ‘**To extract the Invoice Total from the invoice image.**’

**Step 15:** Create a variable through “**Variables**” panel for the “**Get OCR Text**” activity as under:

| Name         | Variable Type | Scope                                                                                                      | Default |
|--------------|---------------|------------------------------------------------------------------------------------------------------------|---------|
| InvoiceTotal | String        | Sequence - ‘This is the code to read a scanned image of an invoice and store extracted data in .CSV file.’ |         |

**Step 16:** Declare the variable **InvoiceTotal** in the “**Output**” Property of “**Get OCR Text**” activity.

**Step 17:** Drag the **“Get OCR Text”** activity from the activity panel and drop it in the workflow.

**Step 18:** Name the **“Get OCR Text”** activity as **Get OCR Text – ‘To extract the e-mail address from Invoice Image.’**

**Step 19:** Create a variable through **“Variables”** panel for the **“Get OCR Text”** activity as under:

| Name         | Variable Type | Scope                                                                                                      | Default |
|--------------|---------------|------------------------------------------------------------------------------------------------------------|---------|
| EmailAddress | String        | Sequence - ‘This is the code to read a scanned image of an invoice and store extracted data in .CSV file.’ |         |

**Step 20:** Declare the variable **EmailAddress** in the **“Output”** Property of **“Get OCR Text”** activity.

**Step 21:** Drag the **“Get OCR Text”** activity from the activity panel and drop it in the workflow.

**Step 22:** Name the **“Get OCR Text”** activity as **Get OCR Text – ‘To extract the Sub total from Invoice Image.’**

**Step 23:** Create a variable through **“Variables”** panel for the **“Get OCR Text”** activity as under:

| Name     | Variable Type | Scope                                                                                                      | Default |
|----------|---------------|------------------------------------------------------------------------------------------------------------|---------|
| SubTotal | String        | Sequence - ‘This is the code to read a scanned image of an invoice and store extracted data in .CSV file.’ |         |

**Step 24:** Declare the variable **SubTotal** in the **“Output”** Property of **“Get OCR Text”** activity.

**Step 25:** Drag the **“Get OCR Text”** activity from the activity panel and drop it in the workflow.

**Step 26:** Name the **“Get OCR Text”** activity as **Get OCR Text – ‘To extract the tax details from Invoice Image.’**

**Step 27:** Create a variable through **“Variables”** panel for the **“Get OCR Text”** activity as under:

| Name | Variable Type | Scope                                                                                                      | Default |
|------|---------------|------------------------------------------------------------------------------------------------------------|---------|
| Tax  | String        | Sequence - ‘This is the code to read a scanned image of an invoice and store extracted data in .CSV file.’ |         |

**Step 28:** Declare the variable **Tax** in the **“Output”** Property of **“Get OCR Text”** activity.

**Step 29:** Drag the **“Build Data Table”** activity from the activity panel.

**Step 30:** Name as the **“Build Data Table”** activity as **Build Data Table - ‘To insert the result in it.’**

**Step 31:** Create a variable through **“Variables”** panel for the **“Build Data Table”** activity as under:

| Name     | Variable Type | Scope                                                                                                      | Default |
|----------|---------------|------------------------------------------------------------------------------------------------------------|---------|
| ResultDT | DataTable     | Sequence - 'This is the code to read a scanned image of an invoice and store extracted data in .CSV file.' |         |

**Step 32:** Declare the variable **ResultDT** in the “**Output**” Property of “**Build Data Table**” activity.

**Step 33:** Drag the “**Add Data Row**” activity from the activity panel

**Step 34:** Name as the “**Add Data Row**” activity as **Add Data Row - 'To insert the data table row in the .CSV file.'**

**Step 35:** Create a variable through “**Variables**” panel for the “**Add Data Row**” activity as under:

| Name     | Variable Type | Scope                                                                                                      | Default |
|----------|---------------|------------------------------------------------------------------------------------------------------------|---------|
| ResultDT | DataTable     | Sequence - 'This is the code to read a scanned image of an invoice and store extracted data in .CSV file.' |         |

**Step 36:** Declare the variable **ResultDT** in the “**Output**” Property of “**Build Data Table**” activity.

**Step 37:** Drag the “**Write CSV**” activity from the activity panel.

**Step 38:** Name the “**Write CSV**” activity as **Write CSV - 'This block of code will extract the data in .CSV format from the invoice image.'**

**Step 39:** Use the **ResultDT** variable in the input property of “**WriteCSV**” property.

**Step 40:** Insert the “.CSV” file in the file path of “**Write CSV**” property.

## 13 Filling a webform from a true PDF file

**Objective:** Code a Robot to Read a true PDF file from and fill the webform.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” activity.
- ✓ “Read PDF Text” and “Assign Into”.
- ✓ “Open Browser” and “Sequence” activity.
- ✓ “For Each” and “IF” activity.
- ✓ “Sequence, Assign” and “Write Line” activity.
- ✓ “Type Into” and “Click” activity.

### Package Prerequisites:

- UiPath.PDF.Activities = 2.0.1

### Algorithm:

**Step 1:** START.

**Step 2:** Declare the variables as 'ExtraxtedData' , 'Row' , 'SpaceSplit' , 'Counter'.

**Step 3:** Add the Read PDF activity to read the PDF file Use the split methods to extract the specific data.

**Step 4:** Open "<https://forms.gle/JXvbYtN4CKynghdR8>" with the help of open browser activity.

**Step 5:** Add For Each activity to iterate the elements in the PDF files split the items with reference of spaces.

**Step 6:** Add Type Into and Click activity.

**Step 7:** STOP.

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the “**Sequence**” activity from the activity panel and drop it in the workflow.

**Step 4:** Name the “**Sequence**” activity as **Sequence – ‘to Read a true PDF file from and fill the webform.’**

**Step 5:** Drag the “**Read PDF Text**” activity from the activity panel and drop it in the workflow.

**Step 6:** Name the “**Read PDF Text**” activity as **Read PDF Text – ‘to Read the data from PDF file.’**

**Step 7:** Create a variable from the “**Variables**” panel in the “**Sequence**” activity as under:

| Name        | Variable Type | Scope                                                           | Default |
|-------------|---------------|-----------------------------------------------------------------|---------|
| ExtractData | String        | Sequence - 'To Read a true PDF file from and fill the webform.' |         |

**Step 8:** Declare the variable **ExtractData** in the “**Output**” property of “**Read PDF Text**” activity.

**Step 9:** Write the PDF file name **PdftoWeb.pdf** in the “**FileName**” property of “**Read PDF Text**” activity.

**Step 10:** Drag and drop the “**Assign**” activity inside the “**Sequence**” activity.

**Step 11:** Change the “**Assign**” activity name **Assign - 'To split the extracted data on the basis of word "Address" and removing empty spaces and store it in "ExtractedData" variable.'**

**Step 12:** Use the ‘**ExtractData**’ variable in the **TO** box and ‘**ExtractedData.Split({"Address"},StringSplitOptions.RemoveEmptyEntries)(1).Trim**’ in the **value** box.

**Step 13:** Drag and drop the “**Assign**” activity inside the “**Sequence**” activity.

**Step 14:** Change the “**Assign**” activity name **Assign - 'To split the extracted data on the basis of new line and store it in a "Row" variable.'**

**Step 15:** Create a variable from the “**Variables**” panel in the “**Sequence**” activity as under:

| Name | Variable Type | Scope                                                           | Default |
|------|---------------|-----------------------------------------------------------------|---------|
| Row  | String[]      | Sequence - 'To Read a true PDF file from and fill the webform.' |         |

**Step 16:** Declare the **'Row'** variable in the **TO** box and **'ExtraxtedData.Split(Environment.NewLine.ToArray,StringSplitOptions.RemoveEmptyEntries)'** in the **value** box.

**Step 17:** Drag and drop the **"Open Browser"** activity from the activity panel and name it as **Open Browser – 'Opens a specific browser using specified URL'** and inside the URL **text field** write:

**<https://forms.gle/JXvbYtN4CKynghdR8>**

**Step 18:** Right-click on **"Open Browser"** activity select **Annotation > Add Annotation 'This block of code will open the specified browser and maximize it.'**

**Step 19:** Change the name of **"Do"** sequence from the **"Open Browser"** activity as **Do – 'This sequence maximizes the browser window.'**

**Step 20:** Drag the **"Maximize Window"** activity from the activity panel and drop it in the workflow.

**Step 21:** Name the **"Maximize Window"** activity as **Maximize Window – 'To maximize the browser window.'**

**Step 22:** Drag and drop the **"For Each Row"** activity and change the name it as – **For Each - 'To extract the data and enter it in the webform until the condition is fulfilled.'**

**Step 23:** Declare the **Row** variable in **IN** and **Item** in **ForEach** section.

**Step 24:** Change the name of **"Body"** sequence from the **"For Each Row"** activity as **Body - 'Extracting pdf data & entering it in the webform'**

**Step 25:** Drag the **"IF"** activity from the activity panel and drop it in the workflow.

**Step 26:** Name the **"IF"** activity as **If - 'If item do not contain any value, split the data and write in the webform else exit the loop.'**

**Step 27:** Use the predefined argument **'not item is Nothing'** in the condition box of **"IF"** activity.

**Step 28:** Drag the **"Sequence"** activity from the activity panel and drop it in the **"Then"** workflow.

**Step 29:** Name the **"Sequence"** activity as **Sequence - 'This block of code splits the data in row variable and enter it into the webform.'**

**Step 30:** Drag and drop the **"Assign"** activity inside the **"Sequence"** activity.

**Step 31:** Change the **"Assign"** activity name **Assign - 'Splits data in the item on the basis of space.'**

**Step 32:** Create a variable from the **"Variables"** panel in the **"Sequence"** activity as under:

| Name       | Variable Type | Scope                                                           | Default |
|------------|---------------|-----------------------------------------------------------------|---------|
| SpaceSplit | String[]      | Sequence - 'To Read a true PDF file from and fill the webform.' |         |

**Step 33:** Declare the **'SpaceSplit'** variable in the **TO** box and **'Item.Split(" "C)'** in the **value** box.

**Step 34:** Drag the **"Type Into"** activity from the activity panel and drop it in the **"Sequence"** activity.

**Step 35:** Name the **"Type Into"** activity as **Type Into - 'Types the name in the name text box of the webform.'**

**Step 36:** Declare the variable **"SpaceSplit(0)"** in the **"Input"** Property of **"Type Into"** activity.

**Step 37:** Drag the **"Type Into"** activity from the activity panel and drop it in the **"Sequence"** activity.

**Step 38:** Name the **"Type Into"** activity as **Type Into - 'Types the e-mail in the mail text box of the webform.'**

**Step 39:** Declare the variable **"SpaceSplit(2)"** in the **"Input"** Property of **"Type Into"** activity.

**Step 40:** Drag the **"Type Into"** activity from the activity panel and drop it in the **"Sequence"** activity.

**Step 41:** Name the **"Type Into"** activity as **Type Into - 'Types the address in the address text box of the webform.'**

**Step 42:** Declare the variable **"SpaceSplit(4)"** in the **"Input"** Property of **"Type Into"** activity.

**Step 42:** Drag the **"Click"** activity from the activity panel and drop it in the **"Sequence"** activity.

**Step 43:** Name the **"Click"** activity as **Click - 'To click submit button.'**

**Step 44:** Name the **"Click"** activity as **Click - 'To submit another response'**

## 14 Creating list of unique words

**Objective:** Code a Robot to read a word file and then create a list of unique words in an excel sheet.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” and “Word Application Scope” activity.
- ✓ “Do, Read Text Assign” and “For Each” activity.
- ✓ “IF and Assign” activity.
- ✓ “Write Line and IF” activity.
- ✓ “Output Data Table, Build Data Table” and “Add Data Row” activity.
- ✓ “Excel Application Scope” Activity
- ✓ “Append range and Write Range” activity

### Package Prerequisites:

The code comprises of UiPath package and it is vital that they are installed in the studio for the Robot to successfully execute the task.

- UiPath.Word.Activities = 1.3.2
- UiPath.Excel.Activities = 2.6.2

### Algorithm:

**Step 1:** START

**Step 2:** Declare three variables as ‘Word\_Output’, ‘Array\_Words’ & ‘DataTable’

**Step 3:** Assign activity to replace punctuations with space

**Step 4:** Assign activity to split the data line by line and save it in ‘Array\_Words’ variable

**Step 5:** Assign activity to split the same data using spaces and assign it to ‘Array\_Words’ variable

**Step 6:** Do the iteration till the end word until words match and then increase the counter value

**Step 7:** Assign activity to increase the counter value one by one when item equals word

**Step 8:** Do nothing when the counter value goes above 1 else create a Build Data Table activity to create a Data Table variable and a data table

**Step 9:** Add Append Range to append the data if the condition doesn’t match

**Step 10:** Add Write Range activity to write the data

**Step 11:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **“Sequence”** activity as **Sequence – ‘To read a word file and then create a list of unique words in an excel sheet.’**

**Step 5:** Add a **“Comment”** activity from the activity panel and write **// To read a word file and then create a list of unique words in an excel sheet.**

**Step 6:** Drag and drop the **“Word Application Scope”** activity from the activity panel and name it as **Word Application Scope - 'To work on a word file.'**

**Step 7:** Insert the file name **“InputWordFile.docx”** in the **FilePath** property of **“Word Application Scope”** activity.

**Step 8:** Inside the **Sequence - 'To read a word file and replace punctuations with space.'** activity drag the **“Read Text”** activity from the activity panel and name it as: **Read Text - 'To read the word file.'**

**Step 9:** Create a variable through **“Variables”** panel for the **“Read Text”** activity as under:

| Name        | Variable Type | Scope                                                                                      | Default |
|-------------|---------------|--------------------------------------------------------------------------------------------|---------|
| Word_Output | String        | Sequence – ‘To read a word file and then create a list of unique words in an excel sheet.’ |         |

**Step 10:** Declare the variable **Word\_Output** in the **“text”** Property of **“Read Text”** activity.

**Step 11:** Drag and drop the **“Assign”** activity inside the **“Sequence”** activity.

**Step 12:** Name the **“Assign”** activity name **Assign - 'Replace punctuations with space.'**

**Step 13:** Use the **Word\_Output** variable in the **“TO”** box and **Word\_Output.Replace(",","").Replace(".", " ").ToLower** in the **“value”** box.

**Step 14:** Drag and drop the **“Assign”** activity inside the **“Sequence”** activity.

**Step 15:** Name the **“Assign”** activity name **Assign - 'To split the data line by line.'**

**Step 16:** Create a variable through **“Variables”** panel for the **“Assign”** activity as under:

| Name        | Variable Type | Scope                                                                                      | Default |
|-------------|---------------|--------------------------------------------------------------------------------------------|---------|
| Array_Words | String[]      | Sequence – ‘To read a word file and then create a list of unique words in an excel sheet.’ |         |

**Step 17:** Use the **Array\_Words** variable in the **“TO”** box and **Word\_Output.Split({Environment.NewLine},StringSplitOptions.RemoveEmptyEntries)** in the **“value”** box.

**Step 18:** Drag and drop the **“Assign”** activity inside the **“Sequence”** activity.

**Step 19:** Name the **“Assign”** activity name **Assign - 'Splitting the same data using spaces.'**

**Step 20:** Use the **Array\_Words** variable in the **“TO”** box and **Word\_Output.Split({""},StringSplitOptions.None)** in the **“value”** box.

**Step 21:** Drag the **“For Each”** activity inside the **Sequence** activity.

**Step 22:** Name it as **“For Each”** activity as **For Each - 'It will start with first word and compare till the end.'**

**Step 23:** Specify the **String** in the **TypeArgument** property of **“For Each”** activity.

**Step 24:** Declare the **Array\_Words** variable in **IN** and **words** in **ForEach** section.

**Step 25:** Name it as **“Sequence”** activity as: **Sequence - 'comparing all other words by using another loop.'**

**Step 26:** Drag the **“For Each”** activity and Name it as **“For Each”** activity as **For Each - 'It will start with first word and compare till the end.'**

**Step 27:** Specify the **Object** in the **TypeArgument** property of **“For Each”** activity.

**Step 28:** Declare the **Array\_Words** variable in **IN** and **item** in **ForEach** section.

**Step 29:** Name it as **“Sequence”** activity as: **Sequence- 'It check whether the word in first loop equals the word in second loop.'**

**Step 30:** Drag the **“IF”** activity inside the **sequence** activity and name it as **“IF”** activity as **If - 'Item equals word increase the counter.'**

**Step 31:** Use the **item.ToString.Trim = word.Trim** Variable in the condition box of **“IF”** activity.

**Step 32:** Drag the **“Assign”** activity in the **“Then”** box of **“IF”** activity.

**Step 33:** Name it as **“Assign”** as **Assign - 'Increase counter one by one.'**

**Step 34:** Create a variable from the **“Variables”** panel in the **“Assign”** activity as under:

| Name    | Variable Type | Scope                                                         | Default |
|---------|---------------|---------------------------------------------------------------|---------|
| Counter | Int32         | Sequence - 'comparing all other words by using another loop.' | 0       |

**Step 35:** Use the **Counter** variable in the **TO** box and **Counter+1** in the **value** box.

**Step 36:** Drag the **“Write Line”** activity and drag it main body sequence of **“For Each”** activity.

**Step 37:** Name it as **Write Line - 'To check the word and output in counter.'**

**Step 38:** Write the text **word + " " +Counter.ToString** in the text box property of **“Write Line”** activity.

**Step 39:** Drag the **“IF”** activity from the activity panel and name it as: **IF - 'counter's value is greater than 1 do nothing else write it in a excel sheet.'**

**Step 40:** Use the **Counter > 1** property in the condition box of **“IF”** activity.

**Step 41:** Drag the **“Sequence”** activity from the activity panel and drag it **“ELSE”** box of **“IF”** activity.

**Step 42:** Name it as **Sequence - 'To create a data table and fill the data in excel sheet.'**

**Step 43:** Right-click on **“Sequence”** activity select **Annotation > Add Annotation** **“This Block of code will build a data table & will add any found unique words to it. It will then write it in an excel sheet.’**

**Step 44:** Drag the **“Build Data Table”** activity and name it as **Build Data Table - 'To create a datatable and create the variable called DataTable.'**

**Step 45:** Use the **‘DataTable’** variable in the **Output** property of **“Build Data Table”** activity.

**Step 46:** Drag the **“Add Data Row”** activity from the activity panel.

**Step 47:** Name it as **Add Data Row - 'It will add a new row for each unique word.'**

**Step 48:** Use the **DataTable** variable in the **Input** property and **{word}** in the **ArrayRow** property of **Add Data Row** activity

**Step 49:** Drag the **“IF”** activity from the activity panel and name it as **IF - 'Append the results if "TestResult.xlsx" exists else write in the same spreadsheet.'**

**Step 50:** Use the **File.Exists("TestResult.xlsx")** property in the condition box of **“IF”** activity.

**Step 51:** Drag the **“Excel Application Scope”** activity from the activity panel and drag it in the **“Then”** box of IF activity

**Step 52:** Name it as **“Excel Application Scope”** activity as: **Excel Application Scope - 'this activity is used for performing any action on an excel sheet.'**

**Step 53:** Right-click on “Excel Application Scope” activity select **Annotation > Add Annotation** ‘this block of code will append the data to an excel sheet.’

**Step 54:** Use the “TestResult.xlsx” in the **WorkbookPath** property of “Excel Application Scope” activity.

**Step 55:** Name it as **Sequence** activity as: **Sequence - 'To append the data in the excel sheet.'**

**Step 56:** Drag the “Append Range” activity from the activity panel and drag it “Sequence” activity.

**Step 57** Name it as “Append Range” activity as **Append Range – 'It will append the data.'**

**Step 58:** Use the **DataTable** variable in the **DataTable** property and “Sheet1” in the **SheetName** property of **Append Range** activity.

**Step 59:** Drag the “Excel Application Scope” activity from the activity panel and drag it in the “Else” box of IF activity.

**Step 60:** Name it as “Excel Application Scope” activity as **Excel Application Scope - 'this activity is used for performing any action on an excel sheet.'**

**Step 61:** Right-click on “Excel Application Scope” activity select **Annotation > Add Annotation** ‘this block of code will create an excel sheet and write in it.’

**Step 62:** Use the “TestResult.xlsx” in the **WorkbookPath** property of “Excel Application Scope” activity.

**Step 63:** Name it as **Sequence** activity as: **Sequence - 'to write the data in the excel sheet.'**

**Step 64:** Drag the “Write Range” activity from the activity panel and drag it “Sequence” activity.

**Step 65:** Name it as “Write Range” activity as **Write Range - 'This will write the data.'**

**Step 66:** Use the **DataTable** variable in the **DataTable** property, “A1” in the **StartingCell** property and “Sheet1” in the **SheetName** property of **Write Range** activity.

## 15 Extracting and storing the subject of emails

**Objective:** Code a Robot to create a queue in orchestrator and store the subject of emails in a .CSV file.

### Learning Outcomes:

After completion of this exercise you will get familiar with the following:

- ✓ **“Sequence”** activity.
- ✓ **“Get IMAP mail messages”** and **“For Each”**.
- ✓ **“Assign”** and **“Write Line”** activity.
- ✓ **“Add Get Queue Items”** and **“Get Queue Items”** activity.
- ✓ **“Build Data Table”** activity.
- ✓ **“Write CSV”** activity.

### Package Prerequisites:

- **UiPath.Excel.Activities = 2.6.2**
- **UiPath.Mail.Activities = 1.5.0**

### Note:

- **Email and password required.**
- **Orchestrator or bot connection is required.**

There are certain steps that we need to follow to create the connection between the orchestrator and the machine.

To use the facility of orchestrator we need to follow below guideline:

- We need to create Tenant
- To create tenant, we need to have unique email details and credentials.
- Queues work as a container that stores tasks and implement them. For example, consider a ticket line queue where the logic of FIFO (First in First Out) applies.
- There should be dedicated name, email address along with password.

**Algorithm:**

**Step 1:** START

**Step 2:** Declare the variables as 'OutputMails' , 'dt1' , 'Subject' , 'QueueItems'

**Step 3:** Use **Get IMAP** activity for reading the Email from Gmail

**Step 4:** Add **For Each** loop to iterate through all Email Subject getting from Gmail

**Step 5:** Use **Add Queue Item** activity for adding the Subject into Queue

**Step 6:** Use **Get Queue Item** activity for Getting the Subject into Queue

**Step 7:** Build the DataTable for building the New Table so that we could add data to CSV

**Step 8:** Add **For Each** Activity again to Display the Subject of Email into CSV File

**Step 9:** Add **Write CSV** Activity to write all the Subject of Email to CSV File

**Step 10:** STOP

**Step by Step process:**

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the “**Sequence**” Activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **Sequence** – ‘**This is the code to create a queue in orchestrator and store the subject of the email in .CSV.**’

**Step 5:** Drag the “**Get IMAP Mail Messages**” activity from the activity panel and drop it in the workflow.

**Step 6:** Name the **Get IMAP Mail Messages** - ‘**To read mail from specified email host.**’

**Step 7:** Create a variable from the “**Variables**” panel in the “**Get IMAP Mail Messages**” activity as under:

| Name        | Variable Type     | Scope      | Default |
|-------------|-------------------|------------|---------|
| OutputMails | List<MailMessage> | Sequence - |         |

**Step 8:** Declare the variable **OutputMails** in the “**Output**” property of “**Get IMAP Mail Messages**” activity.

**Step 9:** Write the **email** ([XYZ@gmail.com](mailto:XYZ@gmail.com)) and **password** (\*\*\*\*\*) in the ‘**Logon**’ property of “**Get IMAP Mail Messages**” activity.

**Step 10:** Write the **MailFolder** (to target specified email folder e.g. ‘**INBOX**’), **port** (993, if you are using IMAP Mail Messages activity) and **server** (“**smtp.gmail.com**”) ‘**Host**’ property of “**Get IMAP Mail Messages**” activity.

**Step 11:** Declare the email read value **20** in the **Top** property of “**Get IMAP Mail Messages**” activity.

**Step 12:** Drag and drop the “**For Each**” activity and change the name it as – **For Each - 'It is used to read the subject of the email one by one.'**

**Step 13:** Declare the **OutputMails** variable in **IN** and **mail** in **ForEach** section.

**Step 14:** Specify the ‘**System.Net.Mail.MailMessage**’ in the TypeArgument property of “**For Each**” activity.

**Step 15:** Change the name of “**Sequence**” sequence activity as **Sequence - 'To get mail subject and add in the queue.'**

**Step 16:** Drag and drop the “**Assign**” activity inside the “**Sequence**” activity.

**Step 17:** Name the “**Assign**” activity name **Assign - 'To assign subject variable and retrieve subject.'**

**Step 18:** Create a variable from the “**Variables**” panel in the “**Assign**” activity as under:

| Name    | Variable Type | Scope                                                  | Default |
|---------|---------------|--------------------------------------------------------|---------|
| Subject | String        | Sequence - 'To get mail subject and add in the queue.' |         |

**Step 19:** Use the ‘**Subject**’ variable in the **TO** box and ‘**mail.Subject**’ in the **value** box.

**Step 20:** Drag and drop the “**Add Queue Item**” activity from the activity panel and name it as: **Add Queue Item - 'To add mails subjects to the orchestrator queue.'**

**Step 21:** Use the “**LabCourseQ**” in the ‘**QueueName**’ property of “**Add Queue Item**” activity.

**Step 22:** Drag the “**Get Queue Items**” activity from the activity panel

**Step 23:** Name the “**Get Queue Items**” activity as: **Get Queue Items - 'To retrieve the queue item.'**

**Step 24:** In the property panel of “**Get Queue Items**” specified these properties

- I. Set the **QueueItemStates** for all variations.
- II. Use the “**LabCourseQ**” in the ‘**QueueName**’ property of “**Get Queue Items**” activity.

**Step 25:** Drag the “**Build Data Table**” activity from the activity panel inside the main ‘**Sequence**’ activity.

**Step 26:** Name the “**Build Data Table**” activity as: **Build Data Table - 'To define the data in csv file from queue items.'**

**Step 27:** Create a variable from the “**Variables**” panel in the “**Build Data Table**” activity as under:

| Name | Variable Type | Scope                                                         | Default |
|------|---------------|---------------------------------------------------------------|---------|
| dt1  | DataTable     | Sequence – ‘To define the data in csv file from queue items.’ |         |

**Step 28:** Use the ‘**dt1**’ variable in the **DataTable** property of “**Build Data Table**” activity.

**Step 29:** Drag the “**For Each**” activity from the activity panel.

**Step 30:** Name the “**For Each**” activity as: **For Each - 'To add data row until the condition is met.'**

**Step 31:** Create a variable from the “**Variables**” panel in the “**For Each**” activity as under:

| Name       | Variable Type                                               | Scope                                                    | Default |
|------------|-------------------------------------------------------------|----------------------------------------------------------|---------|
| QueueItems | System.Collection.Generic.IEnumerable<UiPath.Core.QueueItem | Sequence - 'To add data row until the condition is met.' |         |

**Step 32:** Use the ‘**QueueItems**’ variable in the **IN** box and ‘**item**’ in the **ForEach** box.

**Step 33:** Specify the ‘**UiPath.Core.QueueItem**’ in the **TypeArgument** property of “**For Each**” activity.

**Step 34:** Name the “**Sequence**” sequence activity as **Sequence - 'To add data row.'**

**Step 35:** Drag and drop the “**Add Data Row**” activity inside the “**Body Sequence**” activity.

**Step 36:** Name the “**Add Data Row**” activity as **Add Data Row - 'To add rows in the data table.'**

**Step 37:** Use the specified argument “**{item.SpecificContent("InSubject").ToString}**” in the ‘**ArrayRow**’ property of “**Add Data Row**” activity.

**Step 38:** Drag the “**Write CSV**” activity from the activity panel.

**Step 39:** Name the “**Write CSV**” activity as **Write CSV - 'This will extract the data in the .CSV format from the Email.'**

**Step 40:** Use the “**Untitled.csv**” file name in the ‘**FilePath**’ property of “**WriteCSV**” activity.

**Step 41:** Use the “**dt1**” variable in the ‘**DataTable**’ property of “**Write CSV**” activity.

## 16 Saving attachment with subject 'Resume' from unread emails

**Objective:** Code a Robot to save the attachment from the unread emails having 'Resume' mentioned in the subject line.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” activity.
- ✓ “Get Outlook mail messages” activity.
- ✓ “For Each” activity.
- ✓ “IF” activity.
- ✓ “Save attachments” activity.

### Package Prerequisites:

- UiPath.Excel.Activities = 2.6.2
- UiPath.Mail.Activities = 1.5.0

### Additional Prerequisites:

- Outlook should be configured on your system.
- The email subject should contain the word 'Resume'.

### Algorithm:

**Step 1:** START.

**Step 2:** Declare the variable as 'ReceivedMail' of type mail message.

**Step 3:** Use **Get Outlook** Activity to read the Emails from the Outlook Email website.

**Step 4:** Add **For Each** Activity to iterate through all the Emails coming from Outlook.

**Step 5:** Add the Condition which Contains 'Resume' in the Subject Line.

**Step 6:** Add **Save Attachment** Activity to save all the attachments.

**Step 7:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the “**Sequence**” Activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **Sequence** – ‘**This is a code to save the attachment from the unread emails having 'Resume' mentioned in the subject line.**’

**Step 5:** Drag the “**Get Outlook Mail Messages**” activity from the activity panel and drop it in the workflow.

**Step 6:** Name the **Get Outlook Mail Messages** - ‘**To read mail from specified “Resume” subject email.**’

**Step 7:** Create a variable from the “**Variables**” panel in the “**Get Outlook Mail Messages**” activity as under:

| Name         | Variable Type      | Scope                                                                                                                    | Default |
|--------------|--------------------|--------------------------------------------------------------------------------------------------------------------------|---------|
| ReceivedMail | List< MailMessage> | Sequence - ‘This is a code to save the attachment from the unread emails having 'Resume' mentioned in the subject line.’ |         |

**Step 8:** Declare the variable ‘**ReceiveMail**’ in the “**Output**” property of “**Get Outlook Mail Messages**” activity.

**Step 9:** Use the “**Inbox**” expression in the ‘**MailFolder**’ property of “**Get Outlook Mail Messages**” activity.

**Step 10:** Drag and drop the “**For Each**” activity and change the name it as – **For Each - 'It is used to read the subject of the email.'**

**Step 11:** Declare the ‘**ReceiveMail**’ variable in **IN** and **mail** in **ForEach** section.

**Step 12:** Specify the ‘**System.Net.Mail.MailMessage**’ in the “**TypeArgument**” property of “**For Each**” activity.

**Step 13:** Change the name of “**Sequence**” sequence activity as **Sequence - 'To get mail subject and add in the queue.'**

**Step 14:** Drag the “**IF**” activity from the activity panel.

**Step 15:** Name the “IF” activity as **IF – 'Condition to check if the email subject contains resume or not.'**

**Step 16:** Use the ‘**mail.Subject.Contains(“Resume”)**’ Variable in the condition box of “IF” activity.

**Step 17:** Drag the “**Save Attachments**” activity from the activity panel and drop it in the “**Then**” workflow.

**Step 18:** Name the “**Save Attachments**” activity as: **Save Attachments - 'To use the download the mail attachment in the specified folder.'**

**Step 18:** Use the “**mail**” in the ‘**Message box**’ property of “**Save attachments**” activity.

**Step 19:** Use the “Attachments” in the ‘**FolderPath**’ property of “**Save attachments**” activity.

## 17 Checking data mismatch using “Try catch” mechanism

**Objective:** Code a Robot to build a data table and then fill the data from the .CSV file. Once filled check for the mismatching columns using the try catch mechanism.

### Learning Outcomes

After completion of this exercise you will get familiar with the following:

- ✓ “Sequence” activity.
- ✓ “Try Catch” activity.
- ✓ “Write CSV” activity.

### Package Prerequisites:

- UiPath.Excel.Activities = 2.6.2

### Algorithm

**Step 1:** START

**Step 2:** Declare the variables as 'Name' , 'Age' , 'DataTable1'.

**Step 3:** Add **try catch** block inside the sequence and input name and wrong format age from the user.

**Step 4:** Add Build Data Table Activity for building table for name and age add data row activity.

**Step 5:** Display message as "Exception : you added an invalid data" in the exception activity panel.

**Step 6:** Write the Data into CSV file.

**Step 7:** STOP

### Step by Step process:

**Step 1:** Open UiPath Studio.

**Step 2:** Create the process and name it.

**Step 3:** Drag the **“Sequence”** Activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **Sequence** – **‘This is a code to build a data table and then fill the data from the .CSV file. Once filled check for the mismatching columns using the try catch mechanism.’**

**Step 5:** Drag the **“Try Catch”** activity from the activity panel.

**Step 6:** Name the **“Try Catch”** activity as: **Try catch – ‘It catches the exception and continue the workflow.’**

**Step 7:** Drag the **“Sequence”** activity from the activity panel inside the **‘Try’** block.

**Step 8:** Name the **“Sequence”** activity as: **Sequence – ‘To get the input and enter the data into datatable.’**

**Step 9:** Drag and Drop **“Input Dialog”** activity and name it as **Input Dialog - ‘Ask user to enter the name as input.’** and write the values as under:

| Title    | Label                     |
|----------|---------------------------|
| Name box | “Please enter your name:” |

**Step 10:** Create a variable through **“Variables”** panel for the **Input Dialog - ‘Ask user to enter the name as input.’** activity as under:

| Name | Variable Type | Scope                                                                                                                                                                     | Default |
|------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Name | String        | Sequence – ‘This is a code to build a data table and then fill the data from the .CSV file. Once filled check for the mismatching columns using the try catch mechanism.’ |         |

**Step 11:** Declare the variable **Name** in the **“Output”** Property of **Input Dialog - ‘Ask user to enter the name as input.’**

**Step 12:** Drag and Drop **“Input Dialog”** activity and name it as **Input Dialog - ‘Ask user to enter the age in wrong format.’** and write the values as under:

| Title   | Label                                                     |
|---------|-----------------------------------------------------------|
| Age box | "Please Enter your age in wrong format<br>i.e. Alphabets" |

**Step 13:** Create a variable through “**Variables**” panel for the **Input Dialog - 'Ask user to enter the age in wrong format.'** activity as under:

| Name | Variable Type | Scope                                                                                                                                                                     | Default |
|------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Age  | String        | Sequence – ‘This is a code to build a data table and then fill the data from the .CSV file. Once filled check for the mismatching columns using the try catch mechanism.’ |         |

**Step 14:** Declare the variable **Age** in the “**Output**” Property of **Input Dialog - 'Ask user to enter the age in wrong format.'**

**Step 15:** Drag the “**Build Data Table**” activity from the activity panel.

**Step 16:** Name as the “**Build Data Table**” activity as **Build Data Table - 'To create a data table.'**

**Step 17:** Create a variable through “**Variables**” panel for the “**Build Data Table**” activity as under:

| Name       | Variable Type | Scope                                                                                                                                                                     | Default |
|------------|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| DataTable1 | DataTable     | Sequence - ‘This is a code to build a data table and then fill the data from the .CSV file. Once filled check for the mismatching columns using the try catch mechanism.’ |         |

**Step 18:** Declare the variable **DataTable1** in the “**Output**” Property of “**Build Data Table**” activity.

**Step 19:** Drag the “**Add Data Row**” from the activity panel.

**Step 20:** Name as the “**Add Data Row**” activity as **Add Data Row - 'To add a row in the datatable.'**

**Step 21:** Select “**System.Exception**” in the exception box property.

**Step 22:** Drag the “**Write CSV**” activity from the activity panel.

**Step 23:** Name the “**Write CSV**” activity as **Write CSV - 'To write the input data into CSV file.'**

**Step 24:** Declare the variable **DataTable1** in the “**Input**” Property and **Untitled.csv** variable in the “**FilePath**” Property of “**Write CSV**” activity.

**Step 25:** Drag the “**Sequence**” activity from the activity panel inside the ‘**Catches**’ block.

**Step 26:** Name as the “**Sequence**” activity as **Sequence - 'To catch the exception and print in the message box.'**

**Step 27:** Drag the “**Message Box**” activity from the activity panel.

**Step 28:** Name as the “**Message Box**” activity as: **Message Box - 'To print the exception message.'**

**Step 29:** In the **Text** property of “**Message Box**” write “**Exception: you added an invalid data in the datatable**”

**Step 30:** Drag the “**Message Box**” activity from the activity panel.

**Step 31:** Name as the “**Message Box**” activity as: **Message Box - 'To print the exception message.'**

**Step 32:** In the **Text** property of “**Message Box**” write “**Exception: Cannot write data in csv as value entered is wrong**”

## 18 Generate monthly expenditure report

**Objective:** Code a Robot in UiPath Studio to generate monthly expenditure report by extracting data from a user email account using Re-framework.

### Learning Outcomes:

After completion of this exercise you will get familiar with the following:

- ✓ **“Re Framework”** activity.
- ✓ **“Comment”** and **“Annotation”**.
- ✓ **“Excel Application Scope”** and **“Sequence”** activity.
- ✓ **“For Each”** and **“For Each Row”** activity to set the conditions.
- ✓ **“IF, Add Queue Item, Write Range”** and **“Read Range”** activity and how to set **variables** in a code using **“Variable”** panel.
- ✓ Display output in **“Write Line”**.

### Note:

- ✓ We use the Re Framework to make our code qualitative and quantitative.
- ✓ We use Re Framework to utilize the best coding practices which consist of standard template including the error and exception handling.
- ✓ The Re Framework help us to make the code more reliable.
- ✓ Importance of Re Framework are as follows:
  - This makes the code structural and consistent
  - This makes the code more reliable and modular
- ✓ Easy Exception and error handling.
- ✓ The details in the steps mentioned below concentrate on the activities related to this specific code.
- ✓ For detailed understanding of “Init” functionalities and “Config file”, please refer the documentation folder of your project.
- ✓ The code has been broken down into 4 different sub processes for easy understanding.

### Prerequisites:

1. Please change the path in the config file and the orchestrator queue name present in the code folder.
2. Orchestrator queue is to be created and the same name should be mentioned in the **“QueueName” Property of “Add Queue Item”(Dispatcher Sequence)**.
3. The subject of the email should be **“Monthly Expenditure”** and the attachment should be in **“.XLS / .XLSX”** format.

## 18.1 Open Outlook (Sequence)

Step by Step process:

**Step 1:** Go to **New** in ribbon section.

**Step 2:** Create a **New sequence** and name it.

**Step 3:** Drag the **“Sequence”** Activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **Sequence** – **‘Open Outlook – ‘This is a code to Open the Outlook and Save the attachments’**

**Step 5:** Right-click on **“Sequence”** activity select Annotation > **Add Annotation ‘Description: Open Outlook using Send Hotkey and save the attachments in a folder that has Subject name as “Monthly Expenditure’**

**Step 6:** Drag the **“Send Hotkey”** activity from the activity panel and drop it in the **“Sequence”** activity.

**Step 7:** Name the **“Send Hotkey”** activity as **Send Hotkey - 'To open run command.'**

**Step 8:** In the **“Send Hotkey”** activity select the **“r”** key from the **“Key”** dropdown menu and select the **Win** key modifier in the **Send hotkey** property.

**Step 8:** Drag the **“Type Into”** activity from the activity panel and drop it in the **“Sequence”** activity.

**Step 9:** Name the **“Type Into”** activity as **Type Into - 'To type outlook in the run command window and press enter.'**

**Step 10:** Declare the variable **“Outlook[k(enter)]”** in the **“Input”** Property of **“Type Into”** activity.

**Step 11:** Drag the **“Get Outlook Mail Messages”** activity from the activity panel and drop it in the workflow.

**Step 12:** Name the **Get Outlook Mail Messages** activity as **Get Outlook Mail Messages - 'To get mail from outlook in the inbox folder.'**

**Step 13:** Create a variable from the **“Variables”** panel in the **“Get Outlook Mail Messages”** activity as under:

| Name        | Variable Type      | Scope                                                                         | Default |
|-------------|--------------------|-------------------------------------------------------------------------------|---------|
| OutlookMail | List< MailMessage> | ‘Open Outlook – ‘This is a code to Open the Outlook and Save the attachments’ |         |

**Step 14:** Declare the variable **‘OutlookMail’** in the **“Output”** property of **“Get Outlook Mail Messages”** activity and enter **10** in the **Top** property.

**Step 15:** Use the **“Inbox”** expression in the **‘MailFolder’** property of **“Get Outlook Mail Messages”** activity and enter **4000** in the **TimeoutMS** property.

**Step 16:** Drag the **“For Each”** activity inside the **Sequence** activity.

**Step 17:** Name it as **“For Each”** activity as **For Each - 'To go thorough each mail in the inbox folder and save the attachments from email that has subject Monthly Expenditure.'**

**Step 18:** Specify the **System.Net.Mail.MailMessage** in the **TypeArgument** property and enter the **OutlookMail** in the **values** property of **“For Each”** activity.

**Step 19:** Declare the **OutlookMail** variable in **IN** and **mail** in **ForEach** section.

**Step 20:** Name it as **Body** activity as **Body - 'This sequence is to check, if the subject contains Monthly Expenditure and save the attachment in the attachment folder.'**

**Step 21:** Drag and drop the **“IF”** activity from the activity panel and name it as **IF - 'To check if the subject contains Monthly Expenditure.'**

**Step 22:** Use the **mail.Subject.Contains("Monthly Expenditure")** Variable in the condition box of **“IF”** activity.

**Step 23:** Drag the **“Sequence”** activity inside the **Then** box of **If** activity.

**Step 24:** Name it as **“Sequence”** activity as **Sequence - 'To save the attachments in the Attachment folder.'**

**Step 23:** Drag the **“Save Attachments”** activity from the activity panel and drop it in the **“Sequence”** workflow.

**Step 24:** Name the **“Save Attachments”** activity as: **Save Attachments - 'To save the attcachment.'**

**Step 25:** Use the **“mail”** in the **‘Message box’** property of **“Save attachments”** activity.

**Step 26:** Use the **“AttachmentPath”** in the **‘FolderPath’** property of **“Save attachments”** activity.

## 18.2 Dispatcher

### Step by Step process:

**Step 1:** Go to **New** in ribbon section.

**Step 2:** Create a **New sequence** and name it.

**Step 3:** Drag the **“Sequence”** Activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **Sequence** – **'This is a code to read all the excel file and add the data in amount column to the orchestrator queue.'**

**Step 5:** Right-click on **“Sequence”** activity select **Annotation > Add Annotation** - **'Description: Code to read all the excel files from a folder. Get all the data from the amount column of all the excel files and add them in the orchestrator queue.'**

**Step 6:** Drag the **“Assign”** activity from the activity panel and drop it in the **“Sequence”** activity.

**Step 7:** Name it as **“Assign”** activity as **Assign - 'To store all the excel files in the variable.'**

**Step 8:** Create a variable from the **“Variables”** panel in the **“Assign”** activity as under:

| Name      | Variable Type | Scope                                                                                                               | Default |
|-----------|---------------|---------------------------------------------------------------------------------------------------------------------|---------|
| ExcelFile | String[]      | Sequence – 'This is a code to read all the excel file and add the data in amount column to the orchestrator queue.' |         |

**Step 9:** Declare the **'Excelfile** variable in the **TO** box and **Directory.GetFiles(AttachmentPath)** in the **value** box.

**Step 10:** Drag the **“For Each”** activity inside the **Sequence** activity.

**Step 12:** Name it as **“For Each”** activity as **For Each - This block of code is to read the data present in excel files.'**

**Step 13:** Specify the **String** in the **TypeArgument** property and enter the **ExcelFile** in the **values** property of **“For Each”** activity.

**Step 14:** Declare the **ExcelFile** variable in **IN** and **item** in **ForEach** section.

**Step 15:** Name it as **“Body”** activity as **Body - 'To validate the file type and performing action accordingly.'**

**Step 16:** Drag the **“IF”** activity from the activity panel inside the sequence activity

**Step 17:** Name it as **“IF”** activity as **IF - 'To validate the file extension.'**

**Step 18:** Use the **Path.GetExtension(item)=".xlsx"** or **Path.GetExtension(item)=".xls"** Variable in the condition box of **"IF"** activity.

**Step 19:** Drag and drop the **"Excel Application Scope"** activity from the activity panel and drag it in the **"IF"** activity.

**Step 20:** Name it as **Excel Application Scope - 'This activity is used for performing any action on an excel sheet.'**

**Step 21:** Insert the **"Item"** name in the **WorkbookPath** property of **"Excel Application Scope"** activity.

**Step 22:** Name the **"Do"** sequence activity as **Do- 'Read the excel sheet and extract the data row by row until the condition is fulfilled.'**

**Step 23:** Drag the **"Read Range"** activity from the activity panel.

**Step 24:** Name it as **"Read Range"** activity as **Read Range - 'Read the data from "Sheet1".'**

**Step 25:** Create a variable from the **"Variables"** panel in the **"Read Range"** activity as under:

| Name                 | Variable Type | Scope    | Default |
|----------------------|---------------|----------|---------|
| DataTableTransaction | DataTable     | Sequence |         |

**Step 26:** Enter **Sheet1** in the **SheetName** property, **""** in the **Range** property and declare the **DataTableTransaction** in the **Output** property of **Read Range** activity.

**Step 27:** Drag and drop the **"For Each Row"** activity inside the **"Excel Application Row"** activity.

**Step 28:** Change the **"For Each Row"** activity name **For Each Row - This block of code is to read the data present in "Sheet1" of all excel file.'**

**Step 29:** Declare the **DataTableTransaction** variable in the **IN** box and **row** in the **ForEach** box.

**Step 30:** Name the **Body** activity as **Body - 'To add the data in amount column to the orchestrator queue.'**

**Step 31:** Drag the **Add Queue Item** activity from the activity panel.

**Step 32:** Name it as **"Add Queue Item"** activity as **Add Queue Item - 'Add the data in the orchestrator queue.'**

**Step 33:** Write the **"ExpenditureQueue"** in the **QueueName** property of **Add Queue Item** activity.

**Step 34:** Declare the values in the **ItemInformation** property of **Add Queue Item** activity.

| Name     | Direction | Type   | Value                  |
|----------|-----------|--------|------------------------|
| InAmount | In        | String | row("Amount").ToString |

## 18.3 Process

### Step by Step process:

**Step 1:** Go to **New** in ribbon section.

**Step 2:** Create a **New sequence** and name it.

**Step 3:** Drag the **“Sequence”** Activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **Sequence – Process Transaction - 'This code is to write the amount in the excel sheet.'**

**Step 5:** Right-click on **“Sequence”** activity select **Annotation > Add Annotation Description: To write the result in the "Sheet1" of "Result.xlsx" in the column A.**

**Step 6:** Drag the **“Write Cell”** activity in the sequence activity from the activity panel.

**Step 7:** Name it as **“Write cell”** activity as **Write Cell - 'To writ the result in the excel file.'**

**Step 8:** In the **“Write Cell”** activity properties panel, write the following values in the property panel as mentioned below:

|              |                                                         |
|--------------|---------------------------------------------------------|
| Cell         | "A"+In_transaction_Number.ToString                      |
| SheetName    | "Sheet1"                                                |
| Text         | in_TransactionItem.SpecificContent("InAmount").ToString |
| WorkbookPath | "Result.xlsx"                                           |

**Step 9:** Drag and drop the **“Write line”** activity from the activity panel and name it as **Write line – ‘To display the text.’**

**Step 10:** Inside the text field write it **in\_TransactionItem.SpecificContent("InAmount").ToString**

## 18.4 Close Outlook

### Step by Step process:

**Step 1:** Go to **New** in ribbon section.

**Step 2:** Create a **New sequence** and name it.

**Step 3:** Drag the “**Sequence**” Activity from the activity panel and drop it in the workflow.

**Step 4:** Name the **Sequence** activity as **Close Outlook - 'This is the code to close the outlook application.'**

**Step 5:** Drag the “**Close Application**” activity from the activity panel

**Step 6:** Name it as “**Close Application**” activity as **Close Application - 'To close the "outlook" application.'**